

A Survey on DRL-Based UAV Communications and Networking: DRL Fundamentals, Applications and Implementations

Wei Zhao¹, Member, IEEE, Shaoxin Cui, Graduate Student Member, IEEE, Wen Qiu², Member, IEEE, Zhiqiang He³, Member, IEEE, Zhi Liu⁴, Senior Member, IEEE, Xiao Zheng⁵, Bomin Mao⁶, Senior Member, IEEE, and Nei Kato⁷, Fellow, IEEE

Abstract—Unmanned aerial vehicles (UAVs) are playing an increasingly pivotal role in modern communication networks, offering flexibility and enhanced coverage for a variety of applications. However, UAV networks pose significant challenges due to their dynamic and distributed nature, particularly when dealing with tasks such as power allocation, channel assignment, caching, and task offloading. Traditional optimization techniques often struggle to handle the complexity and unpredictability of these environments, leading to suboptimal performance. This survey provides a comprehensive examination of how deep reinforcement learning (DRL) can be applied to solve these mathematical optimization problems in UAV communications and networking. Rather than simply introducing DRL methods, the focus is on demonstrating how these methods can be utilized to solve complex mathematical models of the underlying problems. We begin by reviewing the fundamental concepts of DRL, including value-based, policy-based, and actor-critic approaches. Then, we illustrate how DRL algorithms are applied to specific UAV network tasks by discussing from problem formulations to DRL implementation. By framing UAV communication challenges as optimization problems, this survey emphasizes the practical value of DRL in dynamic and uncertain environments. We also explore the strengths of DRL in handling large-scale network scenarios and the ability to continuously adapt to changes in the environment. In addition, future research directions are outlined, highlighting the potential for DRL to further enhance UAV communications and expand its applicability to more complex, multi-agent settings.

Index Terms—Deep reinforcement learning (DRL), unmanned aerial vehicles, drone, UAV-assisted wireless networks, optimization, implementation.

Received 7 December 2024; revised 6 May 2025 and 17 June 2025; accepted 18 June 2025. Date of publication 23 June 2025; date of current version 2 January 2026. (Corresponding authors: Wen Qiu; Zhiqiang He.)

Wei Zhao, Shaoxin Cui, Wen Qiu, Zhiqiang He, and Xiao Zheng are with the School of Computer Science and Technology, Anhui University of Technology, Ma'anshan 243002, China (e-mail: zhaowei@ahut.edu.cn; cuihshaoxin6@gmail.com; clorisqu1@gmail.com; tinyzqh@gmail.com; xzheng@ahut.edu.cn).

Zhi Liu is with the Department of Computer and Network Engineering, The University of Electro-Communications, Chofu 182-8585, Japan (e-mail: liuzhi@uec.ac.jp).

Bomin Mao is with the School of Cybersecurity, Northwestern Polytechnical University, Xi'an 710072, China (e-mail: maobomin@nwpu.edu.cn).

Nei Kato is with the Graduate School of Information Sciences, Tohoku University, Sendai 9808579, Japan (e-mail: kato@it.is.tohoku.ac.jp).

Digital Object Identifier 10.1109/COMST.2025.3581912

I. INTRODUCTION

A. Background and Motivation

THE RAPID advancement of UAVs has significantly expanded their use across various industries, including logistics, surveillance, disaster recovery, and 5/6G wireless communication [1], [2]. UAVs offer unprecedented flexibility in delivering services in remote and inaccessible areas, where traditional infrastructure is either unavailable or insufficient. Their ability to provide on-demand communication networks, especially for areas lacking conventional infrastructure, has positioned UAVs as crucial enablers in wireless communication systems [3].

Despite these advancements, managing UAV communication and networking remains highly challenging due to: (1) dynamic network topology: UAVs are highly mobile, leading to constantly changing network structures [4]. The movement of UAVs necessitates continuous adaptation to varying communication conditions [5]; (2) resource constraints: UAVs have limited battery life, processing power, and bandwidth, which need to be efficiently managed to maintain optimal communication performance [6], [7]; (3) real-time decision making: UAVs must handle tasks such as resource allocation, channel assignment, power control, and task offloading in real-time, under dynamic and uncertain conditions [8]. Addressing these challenges often involves solving complex mathematical optimization problems. For example, determining optimal power allocation among UAVs, selecting the best communication channels, and managing data caching efficiently are all critical problems that require effective optimization strategies [9]. In the context of 5/6G, these tasks become even more complex due to the need to manage dense and heterogeneous networks, as well as coordinate across diverse frequency bands and ultra-reliable low-latency communication services [10].

Traditional methods, such as linear programming and convex optimization, are commonly used to address these problems in communication networks. However, these methods face significant limitations: they are often designed for static or slowly changing environments. They rely on full system knowledge, which is difficult to obtain in fast-changing UAV environments. They struggle to cope with the high-dimensional and non-linear nature of UAV network optimization problems, especially when dealing with the

dynamic demands of 5/6G systems [11]. This is where DRL becomes essential [12], [13]. DRL, which allows an agent to learn optimal actions through interaction with its environment, is well-suited for the dynamic and uncertain nature of UAV networks, particularly in the context of 5/6G. DRL offers a robust solution to real-time optimization problems, enabling UAVs to autonomously manage resources, allocate power, optimize channel assignments, and maintain high-quality communication performance, all while adapting to fluctuating network conditions.

B. DRL Emergence in UAV Systems

As UAVs become integral to communication networks, managing their resources and optimizing network performance require solving complex mathematical problems [26]. These problems include power allocation, channel assignment, task offloading, and data caching. Traditionally, researchers have relied on well-established optimization techniques such as linear programming, convex optimization, and heuristic algorithms to address these challenges. While these methods have proven effective in static or less dynamic network environments, they encounter significant limitations in the rapidly changing and unpredictable settings in UAV networks.

There are some key limitations in traditional optimization methods.

(1) Static assumptions and limited adaptability: Traditional optimization techniques often assume static or semi-static environments, where network parameters such as channel conditions, interference levels, and UAV positions are relatively constant. However, in UAV networks, the topology is highly dynamic, with UAVs frequently changing their positions and communication links being subject to fluctuating conditions. This mobility introduces a significant challenge for methods that require pre-defined or static inputs. Traditional approaches fail to adapt efficiently to dynamic environment, leading to suboptimal performance or the need for frequent re-optimization, which incurs high computational costs [27].

(2) Scalability issues: As the number of UAVs and users in the network increases, the complexity of solving optimization problems grows exponentially. Traditional methods [28] like exhaustive search and branch-and-bound can become computationally intractable as they require exploring all possible combinations of variables. In dense UAV networks with a large number of nodes and communication channels, these methods cannot scale effectively, leading to prohibitive computational delays that are unacceptable in real-time applications.

(3) Lack of real-time decision making: Many traditional optimization techniques, particularly those that rely on deterministic models, are designed for offline analysis and optimization. These methods require extensive system knowledge and often cannot operate in real time. UAV networks, on the other hand, demand rapid decision-making to adapt to changing environmental factors, such as network congestion, battery depletion, or interference. The inability of traditional methods to make quick, real-time decisions under uncertain conditions limits their applicability in UAV networks [29].

(4) Difficulty in handling uncertainty: UAV communication systems operate in highly uncertain environments, with factors like weather conditions, node failures, and unpredictable user demands affecting network performance. Traditional optimization methods often rely on deterministic models or require precise knowledge of system parameters, which is not feasible in real-world UAV networks. For example, methods based on convex optimization [30] typically assume known, fixed parameters, which makes them less effective when dealing with the uncertainty and variability inherent in UAV operations. Additionally, these methods are ill-suited to handling the non-linearities and complex interdependencies between variables that characterize UAV communication problems.

(5) Inflexibility to multi-agent and cooperative scenarios: In many UAV-assisted networks, multiple UAVs must cooperate to achieve common goals, such as optimizing coverage or minimizing network latency. Traditional optimization methods often focus on single-agent scenarios, where a central controller optimizes for a single UAV or a group of UAVs treated as a single entity. These methods do not effectively model the distributed decision-making process required in multi-agent environments, where UAVs must collaborate, share information, and make decentralized decisions in real time [1].

To overcome the limitations of traditional optimization techniques in UAV communication networks, DRL has emerged as a powerful tool, which focuses on training an agent to make decisions based on interactions with its environment by reinforcement learning (RL), and excels in handling high-dimensional data and complex, non-linear relationships by deep learning (DL). The integration of these two techniques allows DRL to address the dynamic, uncertain, and large-scale optimization challenges present in UAV networks.

In UAV systems, DRL has proven particularly effective due to its ability to handle dynamic environments, uncertainty, and real-time decision-making. The following factors make DRL well-suited for UAV communication networks with advantages in adaptation to dynamic environments, scalability in complex systems, handling uncertainty, and real-time decision making.

In addition, DRL has advantages in handling sequential decision-making tasks, where an agent learns how to make optimal decisions by interacting with an environment. Many UAV optimization problems can be naturally mapped into the DRL framework, including concepts of states, actions, rewards, and policies. States represent the current status of the UAV network, including channel conditions, battery levels, task queues, or UAV positions. Actions represent the set of possible decisions UAVs can make, such as allocating power, choosing communication channels, caching data, or offloading tasks. Rewards quantify the immediate performance of a decision, such as reducing latency, saving energy, or increasing data throughput. Policy is the strategy the UAV follows in choosing actions to maximize long-term rewards. By structuring optimization problems as an Markov decision process (MDP), DRL allows UAVs to autonomously learn the best course of action based on the current network state and expected future states. This learning-based approach can

TABLE I
EXISTING RELEVANT SURVEYS

Works	DRL Fundamentals	DRL Applications							DRL Implementations	
		PA	CA	CP	TP	EE	TO	Others	RC	CD
[14]	R	✗	✗	✗	✓	✓	✗	✓	✗	✗
[15]	R	✓	✓	✗	✗	✓	✓	✓	✗	✗
[16]	✓	✓	✗	✗	✓	✓	✓	✓	✗	✗
[17]	✗	✓	✓	✗	✓	✓	✓	✓	✗	✗
[18]	✗	✓	✓	✗	✓	✓	✓	✓	✗	✗
[19]	R	✓	✓	✗	✓	✓	✓	✓	✗	✗
[20]	R	✓	✓	✗	✓	✓	✓	✓	✗	✗
[21]	✗	✓	✓	✓	✓	✓	✓	✓	✗	✗
[22]	R	✓	✓	✓	✓	✓	✓	✓	✗	✗
[23]	P	✓	✓	✓	✓	✓	✓	✓	✗	✗
[24]	P	✓	✓	✓	✓	✓	✓	✓	✗	✗
[25]	✓	✓	✓	✓	✓	✓	✓	✓	✗	✗
Ours	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

PA: Power Allocation; CA: Channel Assignment; CP: Caching Optimization; TP: Trajectory Planning; EE: Energy Efficiency; TO: Task Offloading; CD: Constraint Design; RC: Reward consistency; P: Partial; R: Rare.

dynamically adapt to changing network conditions, allowing for real-time optimization.

C. Relevant Surveys and Our Contributions

In this survey, we aim to provide a comprehensive review of how DRL can be applied to solve key optimization problems in UAV communications and networking. Existing related surveys mainly provide general overviews of DRL applications in areas such as power allocation, channel assignment, task offloading, and others, as summarized in Table I. **The core focus of this survey is not just to describe DRL algorithms but to emphasize how DRL can be employed to address specific optimization models within UAV systems, which is the first survey to discuss DRL implementation for optimization models, to the best of our knowledge.**

Our key contributions are as follows:

- *In-depth Coverage of DRL Fundamentals:* We provide a thorough introduction to the fundamental concepts and algorithms of DRL, including value-based, policy-based, and actor-critic methods. This section serves as a primer for readers to understand how these core DRL techniques work and how they can be adapted to address specific optimization problems in UAV communication networks. Unlike existing survey papers that categorize RL methods, our approach focuses on tracing the evolutionary trajectory of RL algorithms, helping readers understand why various algorithms have emerged rather than merely listing them.
- *Implementation of DRL Techniques:* For each major UAV optimization problem, such as trajectory optimization and power allocation, we review different DRL techniques most suited to solving them. We explain how specific DRL algorithms, such as value-based, policy-based, and actor-critic methods, can be adapted to address the unique challenges posed by these optimization problems.
- *Future Directions:* Finally, we identify emerging trends and future research directions where DRL is likely to play a pivotal role in UAV communications and networking.

D. Organization of the Survey

The remainder of this survey is organized as follows: We begin by introducing the core DRL concepts and algorithms, laying the foundation for understanding how DRL techniques operate and how they are applied in various optimization tasks. Following this, we explore the use of DRL in addressing specific optimization challenges in UAV communication networks, including power allocation, channel assignment, caching, and task offloading. For each problem, we examine the relevant DRL techniques, such as value-based, policy-based, actor-critic, and multi-agent methods, demonstrating how they are tailored to solve the mathematical optimization models underlying UAV networks. The survey also discusses emerging applications of DRL in other aspects of UAV communication, as well as future research directions where DRL can further enhance UAV system performance. We conclude by summarizing the key insights and providing a road-map for future advancements in DRL-based UAV communications and networking. The structure of this paper is shown in Fig. 1. Each DRL paradigm has been extensively explored in various application scenarios. Due to the large volume of related works, we choose to focus each section on a representative research topic for each DRL category. This allows us to present a focused, yet detailed, discussion of the system modeling, DRL algorithm design, and implementation in each case. The selected topics are chosen based on their prevalence and significance in the literature and their suitability as representative use cases for the corresponding DRL approaches.

II. A TUTORIAL OF DRL CONCEPTS AND ALGORITHMS

In this section, we will introduce the fundamental concepts of DRL by tracing the evolution of key algorithms as illustrated in Fig. 2. We will begin with Markov decision processes (MDPs) as the foundation, followed by tabular methods based on value functions. As state spaces become larger, the use of value function approximation with neural networks becomes necessary, leading to the development of deep RL. To handle large action spaces, policy-based methods are introduced, which then evolve into Actor-Critic methods by integrating value functions for improved stability. Finally, we will extend

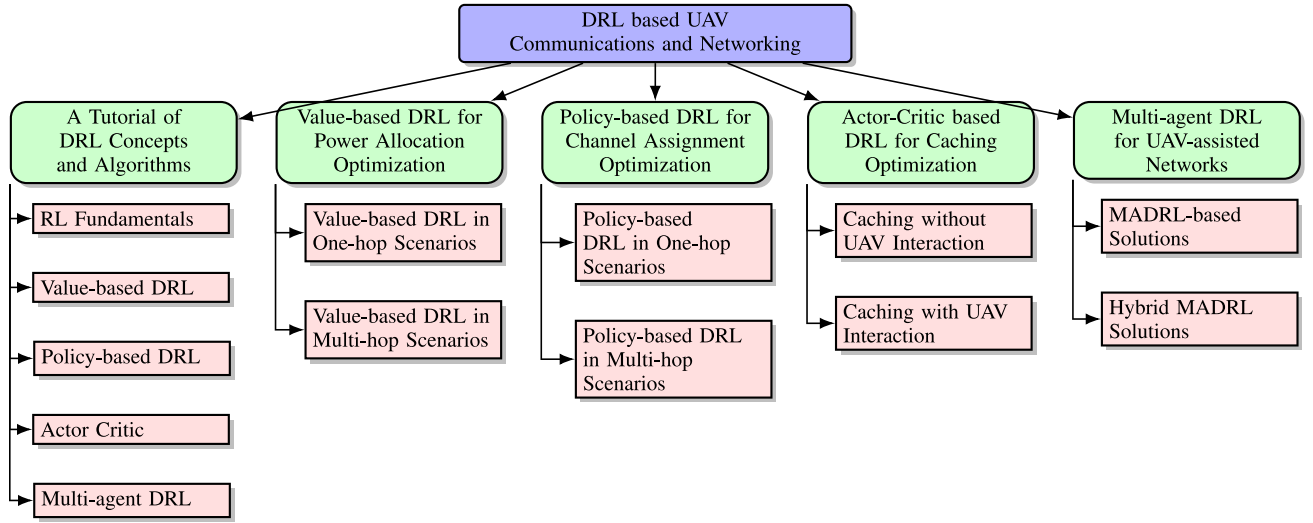


Fig. 1. Survey structure of DRL-based UAV Communications and Networking.

these concepts to multi-agent RL (MARL) to address more complex interaction scenarios.

A. RL Fundamentals

1) *Overview*: In the field of communication, optimization problems are frequently represented as sequential decision-making problems. This is because of the constantly changing environment, such as variations in wireless channels and fluctuations in network load. Additionally, decisions made in this domain have a significant impact on future outcomes, such as scheduling affecting latency and packet loss. Furthermore, optimization in this field is often driven by feedback, with base stations and schedulers adjusting their strategies based on observed performance. Due to the inherent uncertainty in this domain, it is important to incorporate probabilistic uncertainty distributions into the MDP framework. This allows for a more adaptive and accurate representation of the system's behavior.

The MDP is a model that represents the interaction between an agent and its environment, following the Markov property. This means that future states are solely dependent on the current state and action. Once a problem is formulated as an MDP, RL can be utilized as a powerful tool for learning optimal policies through interaction and reward feedback, as shown in Fig. 3. Successfully solving MDPs with RL requires a thorough understanding of key concepts such as the Markov decision framework, Bellman equations, and iterative optimization methods (e.g., policy and value iteration).

2) *Markov Decision Process*: An MDP is formally defined by a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$, providing a structured framework for sequential decision-making in communication systems [75]. Many optimization problems in this domain, such as power control, spectrum allocation, and network scheduling, involve dynamic environments and uncertainty, making them naturally suited for MDP modeling.

In this context, the state space $\mathcal{S}$ captures key system conditions (e.g., channel quality, network load), while the action space $\mathcal{A}$ represents control decisions (e.g., power levels, spectrum selection). The state transition function $\mathcal{P} : \mathcal{S} \times \mathcal{A} \rightarrow$

$\Delta(\mathcal{S})$ models system dynamics, incorporating uncertainties like wireless fading and traffic variations. $\Delta(\mathcal{S})$ denotes the space of probability distributions over $\mathcal{S}$. Specifically, at time step t , given the current state s_t and action a_t , the probability of transitioning to the next state s_{t+1} is defined as $p(s_{t+1} | s_t, a_t)$. The reward function $\mathcal{R} : \mathcal{S} \times \mathcal{A} \rightarrow [0, R_{\max}]$ encodes optimization objectives, such as maximizing throughput or minimizing latency. The discount factor $\gamma \in [0, 1)$ determines how much the agent values future rewards compared to immediate ones.

By formulating communication problems as MDP, RL-based approaches can adaptively learn optimal policies without explicit system models, outperforming traditional optimization techniques in dynamic environments. The agent interacts with the system via a policy π , which can be either deterministic as $\pi : \mathcal{S} \rightarrow \mathcal{A}$ where $a_t = \pi(s_t)$, or stochastic as $\pi : \mathcal{S} \rightarrow \Delta(\mathcal{A})$ where actions are sampled according to $a_t \sim \pi(s_t)$. These sequential interactions generate a trajectory $\tau_\pi = (s_1, a_1, r_1, s_2, \dots, s_{H+1})$, where H is the length of trajectory. The initial state distribution is defined as $\rho : \mathcal{S} \rightarrow \Delta(\mathcal{S})$, completing the MDP description as a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma, \rho)$. The agent seeks a policy that maximizes the expected cumulative reward $J(\pi)$, as defined in eq. (1),

$$J(\pi) = \mathbb{E} \left[\sum_{t=1}^{\infty} \gamma^{t-1} r_t \mid \pi, s_1 \right]. \quad (1)$$

When the maximum expected rewards for all states in $\mathcal{S}$ are known and the environment model is available, the optimal policy can be derived through greedy action selection: at each state s_t , the action that leads to the next state s_{t+1} with the highest expected reward should be selected. A core aspect of reinforcement learning is value function estimation. Three primary approaches exist [12]: dynamic programming, which requires a full environment model; Monte Carlo methods, which estimate values through sampling; and temporal difference (TD) methods, which enable incremental updates without a complete model. The next section addresses two key tasks:

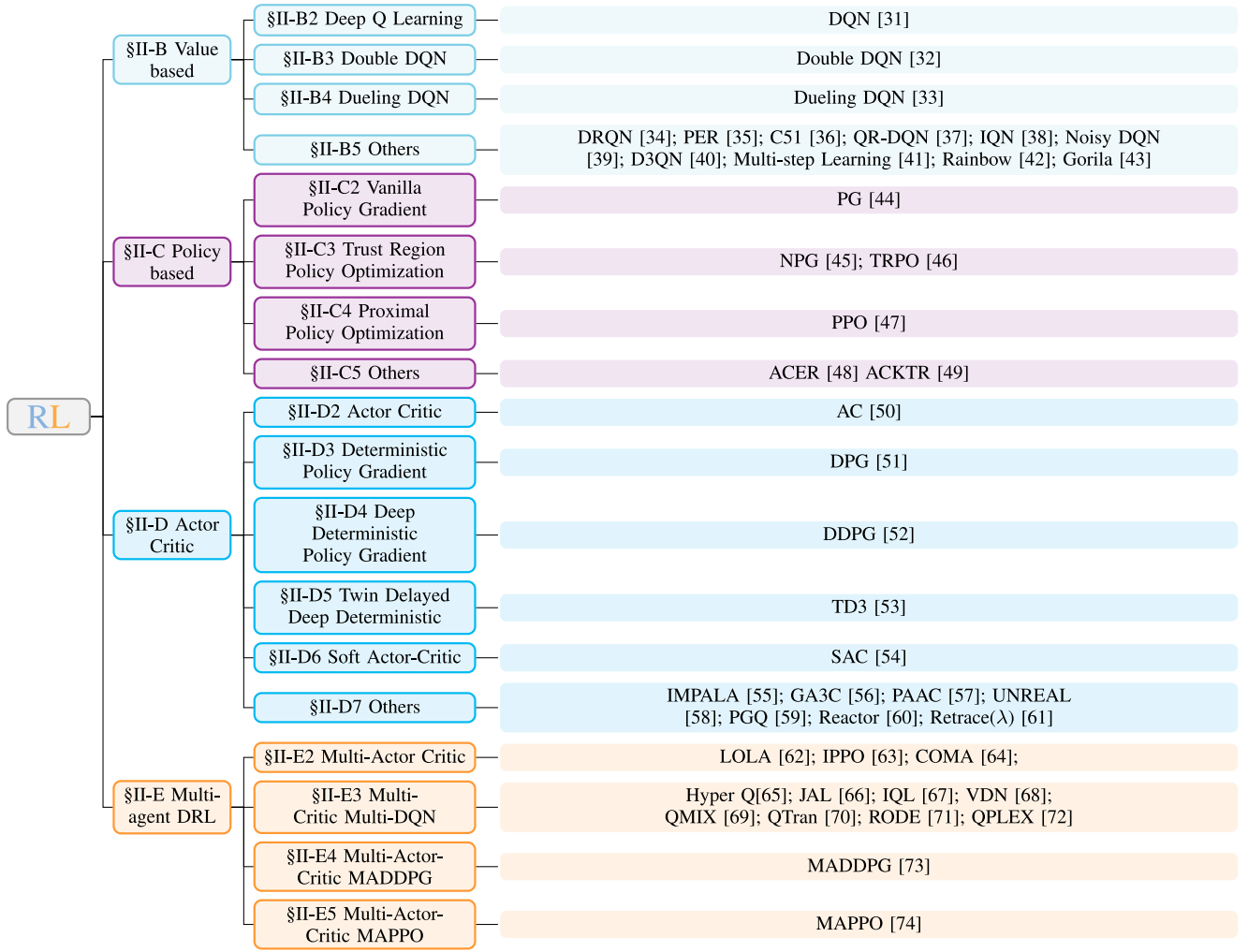


Fig. 2. The taxonomy of DRL algorithms. Policy-based methods are not strictly devoid of critics, but policy-based approaches focus more on policy optimization, while Actor Critic methods emphasize systematic optimization, involving more efficient and parallel sampling.

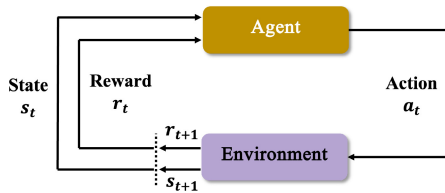


Fig. 3. Process of agent and environment interaction.

evaluating state value functions for a given policy π and finding the globally optimal state value function.

3) *Policy Evaluation and Bellman Equations:* In RL, evaluating a given policy π involves computing the state value function $V^\pi(s_1) : \mathcal{S} \rightarrow \mathbb{R}$ (eq. (2)), which quantifies the expected cumulative discounted rewards from an initial state s_1 under π ,

$$V^\pi(s) = \mathbb{E} \left[\sum_{t=1}^{\infty} \gamma^{t-1} r_t \mid \pi, s = s_1 \right]. \quad (2)$$

To measure the impact of specific actions, the action-state value function $Q^\pi : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ (eq. (3)) quantifies

the expected returns from state s , taking action a , and then following policy π ,

$$Q^\pi(s, a) = \mathbb{E} \left[\sum_{t=1}^{\infty} \gamma^{t-1} r_t \mid \pi, s = s_1, a = a_1 \right]. \quad (3)$$

The value function V^π and action-value function Q^π are recursively linked by the Bellman equations (eq. (4)), which define their relationship for any state $s_t \in \mathcal{S}$ and action $a_t \in \mathcal{A}$ based on dynamic programming principles.

$$V^\pi(s_t) = Q^\pi(s_t, \pi(s_t)),$$

$$Q^\pi(s_t, a_t) = r(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1} \sim p(s_t, a_t)} [V^\pi(s_{t+1})]. \quad (4)$$

For finite-state MDPs, the recursive equations in eq. (4) iteratively compute $V^\pi(s)$ and $Q^\pi(s, a)$. Repeating these updates until convergence provides accurate value estimates, quantifying the effectiveness of policy π across states and actions. Once the value functions are accurately estimated, the policy can be updated by selecting actions that maximize the value function, ensuring the highest expected future reward.

4) *Bellman Optimality Equations:* An optimal policy π^* maximizes the value function $V^\pi(s)$ for all states $s_t \in \mathcal{S}$

TABLE II
ACRONYMS AND ABBREVIATIONS

Acronym	Full form
UAV	unmanned aerial vehicle
DRL	deep reinforcement learning
(MA)RL	(multi-agent) reinforcement learning
DL	deep learning
(PO)MDP	(partially observable) Markov decision process
TD	temporal difference
(M/D)DQN	(mutual/double) deep Q-network
KL	kullback-leibler
FIM	fisher information matrix
TRPO	trust region policy optimization
(MA)PPO	(multi-agent) proximal policy optimization
GAE	generalized advantage estimation
(S)AC	(soft) actor critic
A3C	asynchronous advantage actor critic
A2C	advantage actor critic
(MA/D)DPG	(multi-agent/deep) deterministic policy gradient
TD3	twin delayed deep deterministic
Dec-POMDP	decentralized partially observable MDP
POMG	partially observable Markov games
CTDE	centralized training with decentralized execution
IQL	independent Q-learning
QoS	quality of service
(PD-)NOMA	(power-domain) non-orthogonal multiple access
(A)BS	(aerial) base station
RIS	reconfigurable intelligent surface
CUAV	cognitive radio technology-enhanced UAV
MBA	multiple beam antennas
CSI	channel state information
SINR	signal to interference plus noise ratio
MINLP	mixed-integer nonlinear programming
RB	resource block
UE	user equipment
US	group of user
LoS	line-of-sight
IIoT	industrial Internet of things
(A)AoI	(average) age of information
AAIoTT	aerial access Internet of things network
HAP	high-altitude platform
IoTds	Internet of things devices
JAPORA	joint IoT association, partial offloading, and communication resource allocation
JDCCO	joint data caching and computation offloading

and the action-value function Q^π for all state-action pairs $(s_t, a_t) \in \mathcal{S} \times \mathcal{A}$ [75]. The relationship is defined by the Bellman optimality equations in eq. (5),

$$V^{\pi^*}(s_t) = \max_{a_t \in \mathcal{A}} Q^{\pi^*}(s_t, a_t),$$

$$Q^{\pi^*}(s_t, a_t) = r(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1} \sim p(s_t, a_t)} [V^{\pi^*}(s_{t+1})]. \quad (5)$$

The optimal value function $V^{\pi^*}(s_t)$ represents the maximum expected return from state s_t , while $Q^{\pi^*}(s_t, a_t)$ denotes the highest return achievable by taking action a_t . The optimal policy π^* is then obtained through greedy action selection (eq. (6)),

$$\pi^*(s_t) = \arg \max_{a_t \in \mathcal{A}} Q^{\pi^*}(s_t, a_t), \forall s_t \in \mathcal{S}. \quad (6)$$

Solving the problem requires estimating the value function, reducing it to solving the Bellman optimality equations.

However, directly computing the optimal policy from eq. (6) is challenging as it depends on Q^{π^*} . For tabular MDPs, this is addressed via policy iteration, which alternates between policy evaluation and policy improvement, or value iteration, which iteratively applies the Bellman update until convergence.

5) *Policy Iteration*: The policy iteration algorithm starts with an arbitrary policy π_0 and iteratively performs policy evaluation and policy improvement. The policy $\pi(a_t | s_t)$ is evaluated using eq. (2) and eq. (3), interpreted as probabilities. The value functions V and Q are then updated as in eq. (7),

$$V_{k+1}(s_t) = \sum_{a_t \in \mathcal{A}} \pi_{k+1}(a_t | s_t) \left(r_t + \gamma \sum_{s_{t+1} \in \mathcal{S}} \mathcal{P}_{s_{t+1} | s_t, a_t} V_k(s_{t+1}) \right),$$

$$Q_{k+1}(s_t, a_t) = \sum_{s_{t+1} \in \mathcal{S}} p(s_{t+1} | s_t, a_t) [r_t + \gamma V_k(s_{t+1})]. \quad (7)$$

where $k+1$ represents the next round evaluation based on the k -th policy. The policy in the $k+1$ -th round can be obtained by policy improvement in eq. (8),

$$\pi_{k+1}(s_t) = \arg \max_{a_t \in \mathcal{A}} Q_k(s_t, a_t). \quad (8)$$

For any $s_t \in \mathcal{S}$ and $a_t \in \mathcal{A}$, it follows that deduce $V_k(s_t) = \sum_{a_t \in \mathcal{A}} \pi_k(a_t | s_t) Q_k(s_t, a_t) \leq \sum_{a_t \in \mathcal{A}} \pi_{k+1}(a_t | s_t) Q_k(s_t, a_t)$. By expanding this inequation, we obtain eq. (9) for any $s_t \in \mathcal{S}$ and $a_t \in \mathcal{A}$:

$$Q_k(s_t, a_t) = \mathbb{E}_{p,r} \left[\sum_{t=1}^{\infty} \gamma^{t-1} r_t + \gamma^t V_k(s_t) \right]$$

$$= \mathbb{E}_{p,r} \left[\sum_{t=1}^{\infty} \gamma^{t-1} r_t + \gamma^t \sum_{a_t \in \mathcal{A}} \pi_k(a_t | s_t) Q_k(s_t, a_t) \right]$$

$$\leq \mathbb{E}_{p,r} \left[\sum_{t=1}^{\infty} \gamma^{t-1} r_t + \gamma^t \sum_{a_t \in \mathcal{A}} \pi_{k+1}(a_t | s_t) Q_k(s_t, a_t) \right]$$

$$\leq \mathbb{E}_{p,r,\pi_{k+1}} \left[\sum_{t=1}^{\infty} \gamma^{t-1} r_t \right] = Q_{k+1}(s_t, a_t), \quad (9)$$

where $\mathbb{E}_{p,r}$ represents the expectation of state transitions and rewards. This iterative process monotonically improves the policy across all states until the optimal policy π^* is reached, effectively solving the Bellman equations for optimality.

6) *Value Iteration*: Policy iteration requires full policy evaluation before each improvement step, leading to multiple state space traversals. Value iteration improves efficiency by integrating policy improvement and policy evaluation. Expanding eq. (8) gives eq. (10),

$$\pi_{k+1}(s_t) = \arg \max_{a_t \in \mathcal{A}} Q_k(s_t, a_t)$$

$$= \arg \max_{a_t \in \mathcal{A}} \sum_{s_{t+1}} p(s_{t+1} | s_t, a_t) [r_t + \gamma V_k(s_{t+1})]. \quad (10)$$

Incorporating eq. (10) into eq. (7) leads to the value iteration formula in eq. (11),

$$V_{k+1}(s_t) = \max_{a_t \in \mathcal{A}} \sum_{s_{t+1}} p(s_{t+1} | s_t, a_t) [r_t + \gamma V_k(s_{t+1})]. \quad (11)$$

The corresponding iterative equation for the action value function is expressed in eq. (12),

$$Q_{k+1}(s_t, a_t) = \max_{a_{t+1}} \sum_{s_{t+1}} p(s_{t+1} | s_t, a_t) [r_t + \gamma Q_k(s_{t+1}, a_{t+1})]. \quad (12)$$

In value iteration, the value function estimates are refined by minimizing TD errors from observed trajectories. The policy improves simultaneously through greedy action selection based on the updated value functions. This interleaved approach of policy evaluation and improvement, rather than completing each step sequentially, enhances computational efficiency and facilitates faster convergence.

In value iteration, the current value function is updated using next-state values. However, estimating $V_{s_{t+1}}$ is challenging due to uncertain state transitions in real systems. Since the action space is typically fixed, algorithms often rely on state-action value functions. Based on eq. (12) and discrete sampled data $(s_t, a_t, r_t, s_{t+1}, a_{t+1}) \in \tau_\pi$, the SARSA algorithm update rule is derived as shown in eq. (13),

$$Q_{k+1}(s_t, a_t) \leftarrow Q_k(s_t, a_t) + \alpha(t)(Q_{\text{target}} - Q_k(s_t, a_t)), \\ Q_{\text{target}} = r_t + \gamma Q_k(s_{t+1}, a_{t+1}). \quad (13)$$

The learning rate schedule $\alpha(t)$ controls update magnitudes in RL, while the target value Q_{target} combines the immediate reward r_t with the discounted future reward for a more accurate estimate than $Q_k(s_t, a_t)$. SARSA computes Q_{target} using the action a_{t+1} from the current policy, making it an on-policy algorithm. In contrast, off-policy methods like Q-learning use separate policies for experience generation, allowing data reuse. Offline learning, unlike both on-policy and off-policy methods, relies entirely on a fixed dataset without further interaction with the environment, making it immune to exploration-exploitation trade-offs during training. Q-learning employs an ϵ -greedy sampling policy and computes Q_{target} with a greedy policy, updating as shown in eq. (14),

$$Q_{k+1}(s_t, a_t) \leftarrow Q_k(s_t, a_t) + \alpha(t)(Q_{\text{target}} - Q_k(s_t, a_t)), \\ Q_{\text{target}} = r_t + \gamma \max_{a_{t+1} \in \mathcal{A}} Q(s_{t+1}, a_{t+1}). \quad (14)$$

The main difference between Q-learning and SARSA is their method for estimating the target Q-value. There are several approaches available for this estimation, each designed to balance the trade-off between variance and bias in value function estimation. High variance can lead to instability in learning, while high bias may result in suboptimal policies. Methods such as Monte Carlo estimation, TD learning, and bootstrapping techniques adjust this balance by varying the reliance on sampled returns versus estimated value functions.

- 1) SARSA: $Q_{\text{target}} = r_t + \gamma Q(s_{t+1}, a_{t+1})$,
- 2) Q-Learning: $Q_{\text{target}} = r_t + \gamma \max_{a_{t+1}} Q(s_{t+1}, a_{t+1})$,
- 3) Monte Carlo: $Q_{\text{target}} = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$,
- 4) n -step RL: $r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots + r_{t+n} \gamma^n Q(s_{t+n}, a_{t+n})$.

B. Value-Based DRL

1) *Overview:* In RL, Q-learning and SARSA [12], [76] use tabular methods, which become impractical in continuous state-action spaces. While discretization simplifies the problem, it reduces precision and may prevent reaching global optima. To overcome this, parameterized models [13], especially neural networks, are favored for their strong generalization, enabling more efficient value function learning.

This section explores cases where the state space $\mathcal{S}$ is infinite while the action space $\mathcal{A}$ remains finite. To handle high-dimensional states and enable generalization, parametric function approximation, such as neural networks, is used, represented as $V^\theta(s_t) \simeq V^\pi(s_t)$ and $Q^\theta(s_t, a_t) \simeq Q^\pi(s_t, a_t)$, where θ is updated via reinforcement learning. The goal is to minimize the mean-squared error between $Q^\theta(s_t, a_t)$ and $Q^\pi(s_t, a_t)$, defined by the loss function $J(\theta)$. Using stochastic gradient descent, the SARSA algorithm updates θ as shown in eq. (15),

$$\begin{aligned} \theta &\leftarrow \theta - \alpha(t) \frac{\partial J(\theta)}{\partial \theta} \\ &= \theta + \alpha(t) \left(Q^\pi(s_t, a_t) - Q^\theta(s_t, a_t) \right) \frac{\partial Q^\theta(s_t, a_t)}{\partial \theta} \\ &= \theta + \alpha(t) \left(r_t + \gamma Q^\theta(s_{t+1}, a_{t+1}) - Q^\theta(s_t, a_t) \right) \frac{\partial Q^\theta(s_t, a_t)}{\partial \theta}, \end{aligned} \quad (15)$$

where r_t represents the actual reward, and $Q^\theta(s_{t+1}, a_{t+1})$ and $Q^\theta(s_t, a_t)$ are the estimates provided by the function approximator. The target value Q_{target} is typically used instead of the expected cumulative reward Q^π due to the complexity of computing the latter.

The approximation of $Q^\theta(s_t, a_t)$ and $Q^\theta(s_{t+1}, a_{t+1})$ aims for unbiased updates, but stable training requires additional techniques. While methods like experience replay, target networks, and learning rate scheduling help address sample correlation and instability, and network design, batch normalization, and regularization enhance robustness, these are auxiliary techniques rather than the focus of our discussion. The following sections will primarily explore core algorithmic advancements.

2) *Deep Q Learning:* Deep Q-network (DQN) [31] leverages convolutional neural networks for feature representation, achieving state-of-the-art performance in complex game environments. However, policy instability from shifting data distributions poses a challenge. To address this, experience replay stores past experiences $D_t = \{e_1, \dots, e_t\}$, where $e_t = (s_t, a_t, r_t, s_{t+1})$, improving sample efficiency and stabilizing training.

DQN further mitigates instability by introducing a target network, which decouples updates to $Q^\theta(s_t, a_t)$ and $Q^\theta(s_{t+1}, a_{t+1})$. While the primary network updates continuously, the target network updates periodically, reducing bias and improving stability. The DQN loss function at iteration k is given in eq. (16),

$$J_k(\theta_k) = \mathbb{E}_{e_t \sim D_t} \left[\left(r_t + \gamma \max_{a_{t+1}} Q^{\theta^{\text{target}}} (s_{t+1}, a_{t+1}) - Q^{\theta_k}(s_t, a_t) \right)^2 \right], \quad (16)$$

where θ^{target} represents the target network parameters at iteration k , updated by periodically copying θ_k .

3) *Double DQN*: The double deep Q-network (DDQN) addresses the overestimation issue in DQN by decoupling action selection and value evaluation [32]. In DQN, the max operator for the target state-action value couples these two processes, leading to suboptimal action choices and overly optimistic value estimates. Specifically, when looking at the term $\max_{a_{t+1}} Q^{\theta^{\text{target}}} (s_{t+1}, a_{t+1})$ in eq. (16), it becomes apparent that both the selection of the next action a_{t+1} and the evaluation of Q^{target} depend on the same target network parameters θ^{target} . This coupling is a major cause of value overestimation.

DDQN mitigates this by selecting the next action using the current network parameters θ and evaluating it with the target network θ^{target} , as in $Q^{\text{target}} = r_t + \gamma Q^{\theta^{\text{target}}} (s_{t+1}, \arg \max_{a_{t+1} \in \mathcal{A}} Q^{\theta}(s_{t+1}, a_{t+1}))$. This separation mitigates Q-value overestimation, reduces training time, and enhances learning stability, leading to more reliable value estimation and better overall performance [77].

4) *Dueling DQN*: Dueling DQN introduces a novel decomposition of the state-action value function $Q^{\theta_1, \theta_2, \theta_3}(s_t, a_t) = V^{\theta_1, \theta_2}(s_t) + A^{\theta_1, \theta_3}(s_t, a_t)$ [33]. This architecture splits the network into two streams: one approximating the state value function $V(s_t)$ and the other estimating the advantage function $A(s_t, a_t)$. The key insight is that in many situations, it is unnecessary to estimate the value of each action separately. For instance, in gaming environments, knowing whether to move left or right may be sufficient for avoiding collisions without requiring precise value estimates for each action. This observation motivates the dueling architecture's efficiency in learning state values independently from action advantages. The advantage function follows the property that $\mathbb{E}_{a_t \sim \pi(s_t)} [A^{\pi}(s_t, a_t)] = 0$, leading to a refined formulation in $Q^{\theta_1, \theta_2, \theta_3}(s_t, a_t) = V^{\theta_1, \theta_2}(s_t) + (A^{\theta_1, \theta_3}(s_t, a_t) - \frac{1}{|\mathcal{A}|} \sum_{a_{t+1}} A^{\theta_1, \theta_2}(s_t, a_t))$. By explicitly separating the estimation of state values from action advantages, Dueling DQN enables more robust and efficient learning, particularly in scenarios where understanding the relative importance of different actions is crucial for optimal decision-making.

5) *Others*: Many value function approximation techniques in DRL can be applied to communication systems, enhancing efficiency and adaptability.

To handle partially observable environments, models incorporate historical data and LSTMs [34]. For large action spaces, Deep Monte-Carlo methods [78] enable efficient Q-function evaluation.

Structural improvements like prioritized experience replay [35], distributional RL (C51) [36], and Noisy DQN [39] enhance learning stability and exploration. IQN [38] supports risk-sensitive decision-making, while Rainbow [42] integrates multiple optimizations for robust performance.

Training techniques like Pop-Art [79] and Averaged-DQN [80] improve stability and variance reduction. DQfD [81] leverages expert demonstrations, and soft DQN [82] balances exploration-exploitation. Distributed DRL architectures [43] enable large-scale optimization.

C. Policy-Based DRL

1) *Overview*: Value-based RL evaluates value functions across the state space and derives policies to maximize them. While DQN uses neural approximations for infinite state spaces and handles large discrete action spaces, it struggles with continuous actions. Policy-based methods overcome this by directly optimizing policy parameters ω to maximize the objective $J(\omega)$, reformulating eq. (1) into eq. (17),

$$J(\omega) = \mathbb{E} \left[\mu(s_1) \prod_{t=0}^H \gamma^t p(s_{t+1} | s_t, a_t) \pi^{\omega}(a_t | s_t) r(s_t, a_t) \right]. \quad (17)$$

A fundamental property of policy execution emerges when a policy π^{ω} is executed for a sufficient duration. The frequency of visiting each state-action pair (s_t, a_t) stabilizes to fixed proportions, forming a probability distribution. This stationary distribution is mathematically represented in $d^{\pi^{\omega}}(s_1) = \mu(s_1) \prod_{t=0}^H \gamma^t p(s_{t+1} | s_t, a_t) \pi^{\omega}(a_t | s_t)$. Based on this stationary distribution, the optimization objective can be reformulated in eq. (18),

$$J(\pi^{\omega}) = \mathbb{E}_{(s_t, a_t) \sim d^{\pi^{\omega}}(s)} [r(s_t, a_t)] = \mathbb{E}_{(s_t, a_t) \sim d^{\pi^{\omega}}(s)} [V(s_1)], \quad (18)$$

where the policy, parameterized by ω as $\pi^{\omega}(a_t | s_t)$, has a gradient $\nabla_{\omega} J(\pi^{\omega})$ with a complex dependency structure, influenced by both action selection and the induced state distribution. Estimating this distribution shift is challenging due to unknown environment dynamics, driving the development of policy-based algorithms for effective optimization in uncertain settings.

2) *Vanilla Policy Gradient*: Eq. (17) presents a challenge in computing its derivative due to the product of terms involving the policy π^{ω} . To simplify this, Sutton proposed assuming the stationarity of $d^{\pi^{\omega}}$, resulting in a more elegant formulation of the derivative of the objective function. This reformulation eliminates the need to compute the derivative of the state distribution d^{π} , making the computation of the gradient $\nabla_{\omega} J(\pi^{\omega})$ significantly simpler.

The derivation of the policy gradient involves several important steps. First, we express the objective function in terms of the state-action value function $Q^{\pi^{\omega}}$. Then, by carefully manipulating the terms and applying the log-derivative trick, we arrive at the following formulation:

$$\begin{aligned} \nabla_{\omega} J(\pi^{\omega}) &= \nabla_{\omega} \sum_{s_1 \in \mathcal{S}} d^{\pi^{\omega}}(s_1) \sum_{a_t \in \mathcal{A}} Q^{\pi^{\omega}}(s_t, a_t) \pi^{\omega}(a_t | s_t) \\ &\propto \sum_{s_1 \in \mathcal{S}} d^{\pi^{\omega}}(s_1) \sum_{a_t \in \mathcal{A}} Q^{\pi^{\omega}}(s_t, a_t) \nabla_{\omega} \pi^{\omega}(a_t | s_t) \end{aligned}$$

$$\begin{aligned}
&= \sum_{s_1 \in \mathcal{S}} d^{\pi^\omega}(s_1) \sum_{a_t \in \mathcal{A}} \pi^\omega(a_t | s_t) Q^{\pi^\omega}(s_t, a_t) \frac{\nabla_{\omega} \pi^\omega(a_t | s_t)}{\pi^\omega(a_t | s_t)} \\
&= \mathbb{E}_{\pi^\omega} \left[Q^{\pi^\omega}(s_t, a_t) \nabla_{\omega} \ln \pi^\omega(a_t | s_t) \right]. \quad (19)
\end{aligned}$$

Sutton's work [44] established that high-variance gradients arise from reliance on empirical returns. To reduce variance while maintaining unbiasedness, a baseline is subtracted, weighting updates by an advantage estimate instead of raw returns. Variance reduction techniques range from average returns [83] to more advanced methods [84]. The policy gradient, balancing bias and variance, is expressed as $\nabla_{\omega} J(\pi^\omega) = \mathbb{E}[\sum_{t=0}^{\infty} \Psi_t \nabla_{\omega} \log \pi^\omega(a_t | s_t)]$, where Ψ_t varies across different algorithmic approaches:

- 1) $\sum_{t=0}^{\infty} r_t$: represent the total trajectory reward, measuring the cumulative return from the entire episode.
- 2) $\sum_{t'=t}^{\infty} r_{t'}$: capture the reward following action a_t , forming the basis of the REINFORCE algorithm [83].
- 3) $\sum_{t'=t}^{\infty} r_{t'} - b(s_t)$: introduce a baseline-adjusted version, where $b(s_t)$ serves as a variance-reducing baseline function.
- 4) $Q^\theta(s_t, a_t)$: utilize the state-action value function (critic), fundamental to actor-critic architectures [12].
- 5) $A^\theta(s_t, a_t)$: employ the advantage function to measure the relative benefit of action a_t compared to the expected return.
- 6) $r_t + V^\theta(s_{t+1}) - V^\theta(s_t)$: incorporate the TD residual, central to TD learning approaches [12].

3) *Trust Region Policy Optimization*: Vanilla policy gradient methods assume a stationary state-action visitation frequency (s_t, a_t) , requiring resampling after each policy update and preventing sample reuse. As the policy $\pi^\omega(a_t | s_t)$ is generated by a neural network, any parameter change $\Delta\omega$ directly alters the distribution. Natural policy gradient [45] treats the parameter space as a Riemannian manifold, using KL-divergence as a metric and the Fisher information matrix (FIM) to define curvature. This geometric perspective frames probability distributions as points in Riemannian space, with KL-divergence dictating distance and FIM characterizing local curvature. To control drastic policy shifts, natural policy gradient imposes a constraint $D_{\text{KL}}(\pi^\omega(a_t | s_t) | \pi^{\omega+\Delta\omega}(a_t | s_t)) = \epsilon$, ensuring a fixed distance ϵ between consecutive policies. Building on this, trust region policy optimization (TRPO) [46] constrains optimization steps within a region where the surrogate objective remains an accurate approximation of the true cost function. This approach ensures policy improvement with substantial step sizes.

The expected reward under a new policy $\tilde{\pi}$ relative to the old policy π is expressed in eq. (20),

$$\begin{aligned}
&\mathbb{E}_{\tau \sim \tilde{\pi}} \left[\sum_{t=1}^{\infty} \gamma^{t-1} A^\pi(s_t, a_t) \right] \\
&= \mathbb{E}_{\tau \sim \tilde{\pi}} \left[\sum_{t=1}^{\infty} \gamma^{t-1} (r(s_t, a_t) + \gamma V^\pi(s_{t+1}) - V^\pi(s_t)) \right] \\
&= \eta(\tilde{\pi}) + \mathbb{E}_{\tau \sim \tilde{\pi}} \left[\sum_{t=1}^{\infty} \gamma^t V^\pi(s_{t+1}) - \sum_{t=1}^{\infty} \gamma^{t-1} V^\pi(s_t) \right]
\end{aligned}$$

$$\begin{aligned}
&= \eta(\tilde{\pi}) + \mathbb{E}_{\tau \sim \tilde{\pi}} \left[\sum_{t=2}^{\infty} \gamma^{t-1} V^\pi(s_t) - \sum_{t=1}^{\infty} \gamma^{t-1} V^\pi(s_t) \right] \\
&= \eta(\tilde{\pi}) - \mathbb{E}_{\tau \sim \tilde{\pi}} [V^\pi(s_0)] = \eta(\tilde{\pi}) - \eta(\pi). \quad (20)
\end{aligned}$$

Through algebraic manipulation of eq. (20), we obtain the following relationship:

$$\eta(\tilde{\pi}) = \eta(\pi) + \sum_s d^{\tilde{\pi}}(s_1) \sum_{a_t} \tilde{\pi}(a_t | s_t) A^\pi(s_t, a_t), \quad (21)$$

where $d^{\tilde{\pi}}$ represents the discounted visitation frequencies. A key insight is that if we can ensure $\sum_s d^{\tilde{\pi}}(s) \sum_{a_t} \tilde{\pi}(a_t | s_t) A^\pi(s_t, a_t) \geq 0$, then the expected discounted reward under the new policy will consistently improve. This means that if we start with an initial policy π and find a policy $\tilde{\pi}$ that satisfies this condition, we can guarantee that $\eta(\tilde{\pi}) \geq \eta(\pi)$, establishing monotonic improvement. However, the challenge lies in practically ensuring this condition, as $d^{\tilde{\pi}}(s)$ requires sampling from the new policy $\tilde{\pi}$. This involves extensive sampling for each candidate policy, making the approach computationally prohibitive. TRPO addresses this challenge by introducing an approximate form $\mathcal{L}$:

$$\mathcal{L}(\tilde{\pi}) = \eta(\pi) + \sum_s d^\pi(s_1) \sum_{a_t} \tilde{\pi}(a_t | s_t) A^\pi(s_t, a_t). \quad (22)$$

The only difference between eq. (22) and eq. (21) lies in the discounted visitation frequencies $d(s_1)$. Their gradients maintain directional consistency, allowing eq. (22) for policy updates when the difference between $d^{\tilde{\pi}}$ and d^π remains small. TRPO provides a precise quantification of their performance difference. Given that $\mathbb{E}_{a \sim \pi}[A^\pi(s_t, a_t)] = 0$, we have $\mathbb{E}_{\tilde{a} \sim \tilde{\pi}}[A^\pi(s, \tilde{a})] = \mathbb{E}_{(a, \tilde{a}) \sim (\pi, \tilde{\pi})}[A^\pi(s, \tilde{a}) - A^\pi(s, a)] = P(a \neq \tilde{a} | s) \mathbb{E}_{(a, \tilde{a}) \sim (\pi, \tilde{\pi}) | a \neq \tilde{a}}[A^\pi(s, \tilde{a}) - A^\pi(s, a)] \leq \xi \cdot 2 \max_{s, a} |A^\pi(s, a)|$, where $P(a \neq \tilde{a} | s) \leq \xi$ defines a joint distribution $(a, \tilde{a} | s)$. For each timestep t , n_t represents the count of instances where $a_i \neq \tilde{a}_i$ for $i < t$. Then $P(n_t > 0) \leq 1 - (1 - \xi)^t$, as shown in the expression, $\mathbb{E}_{s_t \sim \tilde{\pi}, a_t \sim \tilde{\pi}}[A^\pi(s_t, a_t)] = P(n_t = 0) \mathbb{E}_{s_t \sim \tilde{\pi}, a_t \sim \tilde{\pi} | n_t=0}[A^\pi(s_t, a_t)] + P(n_t > 0) \mathbb{E}_{s_t \sim \tilde{\pi}, a_t \sim \tilde{\pi} | n_t>0}[A^\pi(s_t, a_t)]$.

Similarly, we have $\mathbb{E}_{s_t \sim \pi, a_t \sim \tilde{\pi}}[A^\pi(s_t, a_t)] = P(n_t = 0) \mathbb{E}_{s_t \sim \pi, a_t \sim \tilde{\pi} | n_t=0}[A^\pi(s_t, a_t)] + P(n_t > 0) \mathbb{E}_{s_t \sim \pi, a_t \sim \tilde{\pi} | n_t>0}[A^\pi(s_t, a_t)]$. By subtracting the expressions and defining $\varepsilon = \max_{s, a} |A^\pi(s, a)|$:

$$\begin{aligned}
&\sum_{t=0}^{\infty} \gamma^t \cdot |\mathbb{E}_{s_t \sim \tilde{\pi}, a_t \sim \tilde{\pi}}[A^\pi(s_t, a_t)] - \mathbb{E}_{s_t \sim \pi, a_t \sim \tilde{\pi}}[A^\pi(s_t, a_t)]| \\
&= \sum_{t=0}^{\infty} \gamma^t \cdot P(n_t > 0) |\mathbb{E}_{s_t \sim \tilde{\pi}, a_t \sim \tilde{\pi} | n_t>0}[A^\pi(s_t, a_t)] \\
&\quad - \mathbb{E}_{s_t \sim \pi, a_t \sim \tilde{\pi} | n_t>0}[A^\pi(s_t, a_t)]| \\
&\leq 4\xi (1 - (1 - \xi)^t) \max_{s_t, a_t} |A^\pi(s_t, a_t)| \\
&= \sum_{t=0}^{\infty} \gamma^t \cdot 4\xi \varepsilon (1 - (1 - \xi)^t) \leq \frac{4\xi^2 \gamma \varepsilon}{(1 - \gamma)^2}. \quad (23)
\end{aligned}$$

Therefore, the quantification of performance difference between eq. (22) and eq. (21) is expressed in $|\eta(\tilde{\pi}) -$

$L(\tilde{\pi})| \leq \frac{4\xi^2\gamma\epsilon}{(1-\gamma)^2}$. Therefore, $\eta(\tilde{\pi}) \geq L(\tilde{\pi}) - \frac{4\xi^2\gamma\epsilon}{(1-\gamma)^2}$. Replacing ξ by the total variation divergence, denoted as D_{TV} . Thus, $\max_s D_{TV}(\pi(\cdot | s) || \tilde{\pi}(\cdot | s)) \leq \xi$ and applying $D_{TV}(p|q)^2 \leq D_{TV}(p|q)$, we obtain $\eta(\tilde{\pi}) \geq L_{\pi}(\tilde{\pi}) - C * D_{KL}^{\max}(\pi, \tilde{\pi})$, where $C = \frac{4\epsilon\gamma}{(1-\gamma)^2}$.

The final objective of TRPO optimization is to calculate the overall advantage of state visitation distribution and actions. This is achieved by using importance sampling to account for any discrepancies between the distribution of training data and the actual state distribution of the policy, as shown in eq. (24),

$$\begin{aligned} J(\omega) &= \sum_{s \in \mathcal{S}} d^{\pi}(s_1) \sum_{a_t \in \mathcal{A}} (\tilde{\pi}^{\omega}(a_t | s_t) A^{\pi}(s_t, a_t)) \\ &= \sum_{s \in \mathcal{S}} d^{\pi}(s_1) \sum_{a_t \in \mathcal{A}} \left(\pi(a_t | s_t) \frac{\tilde{\pi}^{\omega}(a_t | s_t)}{\pi(a_t | s_t)} A^{\pi}(s_t, a_t) \right) \\ &= \mathbb{E}_{s \sim d^{\pi}, a_t \sim \pi} \left[\frac{\tilde{\pi}^{\omega}(a_t | s_t)}{\pi(a_t | s_t)} A^{\pi}(s_t, a_t) \right]. \end{aligned} \quad (24)$$

This optimization is constrained by a trust region limiting the KL-divergence between the old policy π and the new policy $\tilde{\pi}$, expressed as $\mathbb{E}_{s \sim d^{\pi}(s)} [D_{KL}(\tilde{\pi}^{\theta}(\cdot | s) || \pi(\cdot | s))] \leq \Delta_{KL}$, where $d^{\pi} = d^{\pi^{\omega}}$ represents the discounted state-visit frequencies of policy π^{ω} , and Δ_{KL} controls the maximum policy change per step.

4) *Proximal Policy Optimization*: Proximal policy optimization (PPO) [47] combines the performance benefits of TRPO with simpler implementation by using a clipped surrogate objective. The core idea is to calculate a probability ratio between the current policy and the behavior policy, defined as $r(\omega) = \frac{\tilde{\pi}^{\omega}(a|s)}{\pi(a|s)}$, where $\tilde{\pi}^{\omega}$ represents the current policy and π denotes the behavior policy. The initial objective function is formulated as $J(\omega) = \mathbb{E}[r(\omega) A_{\pi}(s, a)]$, where $A_{\pi}(s, a)$ is the advantage function. Rather than employing TRPO's KL divergence constraint, PPO introduces a simpler constraint mechanism by restricting $r(\omega)$ to a narrow interval $[1 - \delta, 1 + \delta]$, where δ is a hyperparameter. This leads to the clipped objective function in eq. (25),

$$\begin{aligned} J(\omega) &= \mathbb{E}[\min(r(\omega) A^{\pi}(s_t, a_t), \text{clip}(r(\omega), 1 - \delta, 1 + \delta) A^{\pi}(s_t, a_t))]. \end{aligned} \quad (25)$$

PPO improves sample efficiency by enabling multiple mini-batch updates per epoch, outperforming traditional policy gradient methods. It retains TRPO's advantages while simplifying implementation and enhancing sample complexity. Empirical studies show the clipped PPO variant outperforms its KL-divergence counterpart across benchmarks.

The stochastic policy $\pi^{\omega}(a_t | s_t)$ adapts to action spaces using a categorical distribution for discrete actions and a Gaussian distribution for continuous ones. This flexibility, with support for both discrete and continuous action spaces, makes PPO suitable for spectrum allocation, power control, and scheduling tasks in communication systems.

5) *Others*: The actor-critic algorithm with experience replay algorithm enhances stability and sample efficiency with a stochastic dueling network, truncated importance sampling,

and a novel trust region method [48]. Similarly, the actor-critic using Kronecker-factored trust region approach leverages natural policy gradient and Kronecker-factored trust region optimization for actor-critic training, achieving superior sample efficiency compared to other methods [49].

D. Actor Critic

1) *Overview*: The actor critic (AC) framework combines value-based and policy-based RL methods, where the actor selects actions and the critic evaluates them, balancing the high variance of policy gradients with the bias of value approximation. Compared to policy-based methods, the actor's updates in actor-critic heavily rely on the critic.

2) *Actor Critic*: The AC algorithm [50] consists of a policy function and a value function, using a neural network to approximate state values. Let θ denote the value function parameters and ω represent the policy function parameters. The policy parameter update follows $\omega \leftarrow \omega + \alpha_{\omega}(t) Q^{\theta}(s_t, a_t) \nabla_{\omega} \ln \pi^{\omega}(a_t | s_t)$, where $\alpha_{\omega}(t)$ represents the actor learning rate. The parameters of action-state value are updated according to $\theta \leftarrow \theta + \alpha_{\theta}(t)(r_t(s_t, a_t) + \gamma Q^{\theta}(s_{t+1}, a_{t+1}) - Q^{\theta}(s_t, a_t)) \nabla_{\theta} Q^{\theta}(s_t, a_t)$, where $\alpha_{\theta}(t)$ denotes the critic learning rate.

AC methods offer robust stability and favorable convergence but face practical limitations, leading to enhanced variants like asynchronous advantage AC (A3C) [85], which enables parallel training with multiple actors synchronizing periodically with global parameters. The advantage AC (A2C) [86], a synchronous refinement of A3C, ensures more consistent parameter updates by requiring all actors to complete tasks before global updates, enhancing stability and accelerating convergence.

3) *Deterministic Policy Gradient*: RL policies can be stochastic or deterministic. A stochastic policy samples actions from a probability distribution $\pi(s_t)$, with updates relying on $Q(s_t, a_t)$. However, computing policy gradients requires integration over both state and action spaces, making it computationally expensive, especially in high-dimensional action spaces.

The deterministic policy gradient (DPG) algorithm [51] simplifies this by directly mapping states to actions: $a_t = \pi(s_t)$, eliminating action-space integration. This reduces computational complexity and improves sample efficiency, particularly in large or continuous action spaces. The objective function for the deterministic policy is formulated as $J(\omega) = \int_{\mathcal{S}} d^{\pi}(s) Q^{\theta}(s, \pi^{\omega}(s)) ds$, where $d^{\pi}(s)$ represents the stationary distribution of states under policy π . DPG allows for more efficient estimation of the action-value function compared to its stochastic counterpart. By applying the chain rule to $J(\omega)$, we can derive eq. (26),

$$\begin{aligned} \nabla_{\omega} J(\omega) &= \int_{\mathcal{S}} d^{\pi}(s) \nabla_a Q^{\theta}(s, a) \nabla_{\omega} \pi^{\omega}(s) \Big|_{a=\pi^{\omega}(s)} ds \\ &= \mathbb{E}_{s \sim d^{\pi}} \left[\nabla_a Q^{\theta}(s, a) \nabla_{\omega} \pi^{\omega}(s) \Big|_{a=\pi^{\omega}(s)} \right]. \end{aligned} \quad (26)$$

Deterministic policies offer computational efficiency by evaluating $Q^{\theta}(s, \pi^{\omega}(s))$ directly, avoiding the need for

computing expected values across actions and eliminating importance sampling, which enhances stability. However, deterministic policies limit exploration, especially in low-noise environments. This can be mitigated by adding noise to the policy during action selection or by using an off-policy learning strategy with a stochastic behavior policy for sample collection.

4) *Deep Deterministic Policy Gradient*: Deep deterministic policy gradient (DDPG) [52] extends the DPG framework by using deep neural networks to approximate the policy π^w and critic Q^θ , addressing stability issues in actor-critic methods for complex control problems. To enhance stability, DDPG incorporates DQN techniques like experience replay and target networks and adapts the framework for continuous action spaces. However, the deterministic policy limits exploration. Thus, DDPG adds exploration noise by modifying the policy as $\pi'(s_t) = \pi^w(s_t) + \mathcal{N}$.

A key challenge in value function approximation is computing Q^{target} with the same network, leading to instability. DDPG addresses this with “soft” target updates, instead of direct weight copying as in DQN. By maintaining target policy and state-action value networks, DDPG achieves more stable training. The critic update follows $\theta \leftarrow \theta + \alpha_\theta(t)(y_t - Q^\theta(s_t, a_t))\nabla_\theta Q^\theta(s_t, a_t)$, where the target value is computed as $y_t = r_t(s_t, a_t) + \gamma Q^{\theta^{\text{target}}}(s_{t+1}, \pi^{\omega^{\text{target}}}(s_{t+1}))$. Experience replay, adopted from DQN, stabilizes training by breaking temporal correlations and providing more independent, identically distributed samples. The actor update follows the same form as in DPG, while target networks use a soft update mechanism, $\theta^{\text{target}} \leftarrow \zeta\theta + (1 - \zeta)\theta^{\text{target}}$ and $\omega^{\text{target}} \leftarrow \zeta\omega + (1 - \zeta)\omega^{\text{target}}$, with a slow rate ζ to ensure gradual adjustments, enhancing training stability.

5) *Twin Delayed Deep Deterministic*: Twin delayed deep deterministic (TD3) [53] mitigates overestimation bias in actor-critic architectures by adapting DDQN techniques within the DDPG framework. Although DDQN separates value estimation and action selection, actor-critic models with neural network approximations still face overestimation. To counter this, TD3 applies the current policy instead of the target policy $y_t = r(s_t, a_t) + \gamma Q^{\theta^{\text{target}}}(s_{t+1}, \pi^w(s_{t+1}))$.

Building upon the principles of double Q-learning, TD3 utilizes dual deterministic actors ($\pi^{\omega_1}, \pi^{\omega_2}$) paired with corresponding critics ($Q^{\theta_1}, Q^{\theta_2}$). This architecture results in double Q-learning Bellman targets, as shown in eq. (27),

$$\begin{aligned} y_t^1 &= r(s_t, a_t) + \gamma Q^{\theta_2^{\text{target}}}(s_{t+1}, \pi^{\omega_1}(s_{t+1})), \\ y_t^2 &= r(s_t, a_t) + \gamma Q^{\theta_1^{\text{target}}}(s_{t+1}, \pi^{\omega_2}(s_{t+1})). \end{aligned} \quad (27)$$

While optimizing policy π^w using independent estimation in target updates helps avoid policy-induced bias, the unconstrained critics can lead to situations where estimation of one critic consistently exceeds the other: $Q^{\theta_2}(s_t, \pi^{\omega_1}(s_t)) > Q^{\theta_1}(s_t, \pi^{\omega_1}(s_t))$. TD3 addresses it through the clipped double Q-Learning algorithm, which takes the minimum of both estimation, as shown in $y_1 = r(s_t, a_t) + \gamma \min_{j=1,2} Q_j^{\theta^{\text{target}}}(s_{t+1}, \pi^{\omega_1}(s_{t+1}))$.

In actor-critic models, interdependent policy and value updates can cause value overestimation and degrade policy

quality. TD3 addresses this by updating the policy network less frequently than the Q-function, similar to DQN’s target network approach, ensuring more accurate value estimation. It also mitigates overfitting in deterministic policies by adding clipped noise to actions and averaging over mini-batches, as shown in eq. (28),

$$\begin{aligned} y &= r(s_t, a_t) \\ &+ \gamma \min_{j=1,2} Q_j^{\theta^{\text{target}}}(s_{t+1}, \pi^{\omega_1}(s_{t+1})) \\ &+ \text{clip}(\mathcal{N}(0, \sigma), -c, +c), \end{aligned} \quad (28)$$

where σ and c control the noise characteristics. This approach adds clipped random noise to selected actions and averages over mini-batches. TD3 addresses challenges in continuous action spaces by applying double Q-Learning and stabilization techniques, effectively reducing overestimation bias and ensuring stable training by balancing policy and value network updates.

6) *Soft Actor-Critic*: Soft actor-critic (SAC) [54], [87] is an off-policy actor-critic algorithm based on maximum entropy RL. By integrating policy entropy into the reward, SAC promotes exploration while maintaining performance, optimizing for both expected return and entropy to balance exploration and exploitation. The augmented reward function is $r_{\text{soft}}(s_t, a_t) = r(s_t, a_t) + \beta \mathbb{E}_{s_{t+1} \sim p} [\mathcal{H}(\pi(\cdot | s_{t+1}))]$, where β is a temperature parameter balancing entropy, p is the state transition distribution, and $\mathcal{H}(\pi(\cdot | s_t))$ is the policy entropy, increasing action randomness to promote exploration and prevent premature convergence. d^π is the joint distribution of (s, a) induced by the policy π , and β is a coefficient that balances reward maximization and entropy maximization. Building upon the traditional Bellman equation in eq. (29),

$$Q(s_t, a_t) = r(s_t, a_t) + \mathbb{E}_{\substack{s_{t+1} \sim p \\ a_{t+1} \sim \pi}} [Q(s_{t+1}, a_{t+1})]. \quad (29)$$

The soft Q-function evaluation can be derived in eq. (30),

$$\begin{aligned} Q_{\text{soft}}(s_t, a_t) - r(s_t, a_t) &= \beta \mathbb{E}_{s_{t+1}} [H(\pi(\cdot | s_{t+1}))] + \mathbb{E}_{s_{t+1}, a_{t+1}} [Q_{\text{soft}}(s_{t+1}, a_{t+1})] \\ &= \beta \mathbb{E}_{s_{t+1}} \left[\mathbb{E}_{a_{t+1}} (-\log \pi(a_{t+1} | s_{t+1})) \right] \\ &\quad + \mathbb{E}_{s_{t+1}, a_{t+1}} [Q_{\text{soft}}(s_{t+1}, a_{t+1})] \\ &= \mathbb{E}_{s_{t+1}, a_{t+1}} [Q_{\text{soft}}(s_{t+1}, a_{t+1}) - \beta \log(\pi(a_{t+1} | s_{t+1}))]. \end{aligned} \quad (30)$$

Next, we apply the soft policy improvement process. Based on the soft policy improvement theorem, a new policy $\tilde{\pi}$ is defined as $\tilde{\pi}(a_t | s_t) \propto \exp(\frac{1}{\beta} Q_{\text{soft}}^\pi(s_t, a_t))$, $\forall s_t$. Assume Q and $\int \exp(Q(s_t, a_t))$ are bounded for any state s . Then $Q_{\text{soft}}^{\tilde{\pi}}(s_t, a_t) \geq Q_{\text{soft}}^\pi(s_t, a_t) \forall s_t, a_t$. We can prove this theorem according to the definition of KL divergence, leading to eq. (31),

$$\begin{aligned} -D_{\text{KL}}(\pi(\cdot | s) \parallel \tilde{\pi}(\cdot | s)) &= \int_a \pi(a | s) \log \frac{\tilde{\pi}(a | s)}{\pi(a | s)} da \\ &= \int_a \pi(a | s) \log \tilde{\pi}(a | s) da - \int_a \pi(a | s) \log \pi(a | s) da \end{aligned}$$

$$\begin{aligned}
&= \int_a \pi(a | s) \log \frac{\exp\left(\frac{1}{\beta} Q_{\text{soft}}^\pi(s, a)\right)}{\int_a \exp\left(\frac{1}{\beta} Q_{\text{soft}}^\pi(s, a)\right) da} da + \mathcal{H}(\pi(a | s)) \\
&= \int_a \pi(a | s) \frac{1}{\beta} Q_{\text{soft}}^\pi(s, a) da \\
&\quad - \int_a \pi \left(\log \int_a \exp\left(\frac{1}{\beta} Q_{\text{soft}}^\pi(s, a)\right) da \right) da + \mathcal{H}(\pi(a | s)) \\
&= \mathbb{E}_{a \sim \pi} \left[\frac{1}{\beta} Q_{\text{soft}}^\pi(s, a) \right] \\
&\quad - \log \int_a \exp\left(\frac{1}{\beta} Q_{\text{soft}}^\pi(s, a)\right) da + \mathcal{H}(\pi(a | s)). \quad (31)
\end{aligned}$$

Maintaining the current value estimation $Q_{\text{soft}}^\pi(s, a)$, but acting according to the new policy $\tilde{\pi}$,

$$\begin{aligned}
&\mathcal{H}(\tilde{\pi}(\cdot | s)) + \mathbb{E}_{a \sim \tilde{\pi}} \left[\frac{1}{\beta} Q_{\text{soft}}^\pi(s, a) \right] \\
&= -\text{D}_{\text{KL}}(\tilde{\pi}(\cdot | s) \| \pi(\cdot | s)) \\
&\quad + \log \int \exp\left(\frac{1}{\beta} Q_{\text{soft}}^\pi(s, a)\right) da \\
&= 0 + \log \int \exp\left(\frac{1}{\beta} Q_{\text{soft}}^\pi(s, a)\right) da. \quad (32)
\end{aligned}$$

Since $\text{D}_{\text{KL}}(\tilde{\pi}(\cdot | s) \| \pi(\cdot | s)) \geq 0$ with equality holding only if $\pi = \tilde{\pi}$, we have $\mathcal{H}(\pi(\cdot | s)) + \mathbb{E}_{a \sim \pi} \left[\frac{1}{\beta} Q_{\text{soft}}^\pi(s, a) \right] \leq \mathcal{H}(\tilde{\pi}(\cdot | s)) + \mathbb{E}_{a \sim \tilde{\pi}} \left[\frac{1}{\beta} Q_{\text{soft}}^\pi(s, a) \right]$. Multiplying both sides by β , the relationship between the soft V values under the two policies is obtained as $\beta \mathcal{H}(\pi(\cdot | s)) + \mathbb{E}_{a \sim \pi} [Q_{\text{soft}}^\pi(s, a)] \leq \beta \mathcal{H}(\tilde{\pi}(\cdot | s)) + \mathbb{E}_{a \sim \tilde{\pi}} [Q_{\text{soft}}^\pi(s, a)]$. Then, we can derive eq. (33),

$$\begin{aligned}
&Q_{\text{soft}}^\pi(s, a) \\
&= \mathbb{E}_{s_2} [r_1 + \gamma(\mathcal{H}(\pi(\cdot | s_2)) + \mathbb{E}_{a_2 \sim \pi} [Q_{\text{soft}}^\pi(s_2, a_2)])] \\
&\leq \mathbb{E}_{s_2} [r_1 + \gamma(\mathcal{H}(\tilde{\pi}(\cdot | s_2)) + \mathbb{E}_{a_2 \sim \tilde{\pi}} [Q_{\text{soft}}^\pi(s_2, a_2)])] \\
&= \mathbb{E}_{s_2} [r_1 + \gamma(\mathcal{H}(\tilde{\pi}(\cdot | s_2)) + r_2)] \\
&\quad + \gamma^2 \mathbb{E}_{s_3} [\mathcal{H}(\pi(\cdot | s_3)) + \mathbb{E}_{a_3 \sim \pi} [Q_{\text{soft}}^\pi(s_3, a_3)]] \\
&\quad \vdots \\
&\leq \mathbb{E}_{\tau \sim \tilde{\pi}} \left[r_0 + \sum_{t=1}^{\infty} \gamma^t (\mathcal{H}(\tilde{\pi}(\cdot | s_t)) + r_t) \right] \\
&= Q_{\text{soft}}^{\tilde{\pi}}(s, a). \quad (33)
\end{aligned}$$

Therefore, our task is to find a new policy that satisfies the condition described in eq. (34),

$$\pi_{\text{new}} = \arg \min_{\pi'} \text{D}_{\text{KL}} \left(\pi'(\cdot | s_t) \parallel \frac{\exp(\beta^{-1} Q_{\text{soft}}^{\pi_{\text{old}}}(s_t, \cdot))}{Z^{\pi_{\text{old}}}(s_t)} \right). \quad (34)$$

Eq. (34) is equivalent to eq. (35),

$$\begin{aligned}
&\min_{\omega} \text{D}_{\text{KL}} \left(\pi^\omega(\cdot | s_t) \parallel \exp \left[\beta^{-1} Q^\theta(s_t, a_t) - \log(Z^\theta(s_t)) \right] \right) \\
&= \min_{\omega} \mathbb{E}_{a_t \sim \pi^\omega} \left[\log \pi^\omega(a_t | s_t) - \beta^{-1} Q^\theta(s_t, a_t) + \log(Z^\theta(s_t)) \right] \\
&= \min_{\omega} \mathbb{E}_{a_t \sim \pi^\omega} \left[\beta \log \pi^\omega(a_t | s_t) - Q^\theta(s_t, a_t) \right]. \quad (35)
\end{aligned}$$

Combining clip double Q-learning and experience replay, the loss function of the actor can be written in eq. (36),

$$L_\pi(\omega) = \mathbb{E}_{s_t \sim d} \left[\mathbb{E}_{a_t \sim \pi^\omega} \left[\beta \log \pi^\omega(a_t | s_t) - \min_{\theta_i} Q^{\theta_i}(s_t, a_t) \right] \right]. \quad (36)$$

The temperature parameter β plays a key role in balancing entropy maximization with reward optimization. An improper β can significantly impact performance; for instance, a high β turns the algorithm into a standard AC method, overemphasizing entropy at the cost of reward signals. To address this, SAC introduces an adaptive temperature mechanism with a target entropy $\hat{H}$, acknowledging that optimal temperature varies by task and learning stage. The suggested value for the target entropy, $\hat{H}$, is determined to be (A) , and the temperature is adjusted automatically by optimizing an entropy-based loss function, as defined in eq. (37),

$$\begin{aligned}
L(\beta) &= \mathbb{E}_{\substack{s_t \sim d \\ a_t \sim \pi}} \left[-\beta \log \pi(a_t | s_t) - \beta \hat{H} \right] \\
&= \beta \mathbb{E}_{s_t \sim D} \left[H(\pi(\cdot | s_t)) - \hat{H} \right]. \quad (37)
\end{aligned}$$

The update formula of β is given in eq. (38),

$$\begin{aligned}
\beta &= \beta - \alpha_e(t) \cdot \nabla_{\beta} L(\beta) \\
&= \beta - \alpha_e(t) \mathbb{E}_{s_t \sim D} \left[H(\pi(\cdot | s_t)) - \hat{H} \right], \quad (38)
\end{aligned}$$

where $\alpha_e(t)$ represents a learning rate for the temperature.

7) *Others*: Recent advancements in actor-critic methods have focused on computational efficiency and scalability through distributed architectures and optimized training. The importance-weighted actor-learner architecture [55] demonstrates high scalability with its V-trace off-policy correction, enabling stable learning across thousands of machines. Other architectures, like GPU A3C [56] and parallel advantage AC [57], enhance efficiency through hybrid CPU/GPU approaches and single-machine parallelization. Unsupervised reinforcement and auxiliary learning [58] improves data efficiency with auxiliary tasks. Model-based reinforcement learning improves efficiency and reduces sample complexity by enabling the agent to interact with a learned world model [88].

Hybrid methods address limitations in policy gradients, such as policy gradient and Q-learning [59], which combines policy gradients with Q-learning, and the reactor architecture [60], which improves sample efficiency using multi-step returns. Off-policy corrections, like retrace(λ) [61], combine importance sampling and tree-backup methods for low variance and efficiency, further enhanced in the reactor framework with a β -leave-one-out policy gradient for practical sample efficiency.

E. Multi-Agent DRL

1) *Overview*: While single-agent DRL has seen success, it struggles with complex tasks, driving interest in multi-agent systems that mimic natural collective behaviors like migration or pack hunting. These systems offer cooperation, flexibility, adaptability, and redundancy for improved safety. However, traditional RL methods cannot be directly applied due to challenges in agent interactions and coordination, leading to the development of MARL. Approaches such as multi-actor critic, multi-critic DQN, multi-agent DDPG (MADDPG), and multi-agent PPO (MAPPO) address various multi-agent learning challenges.

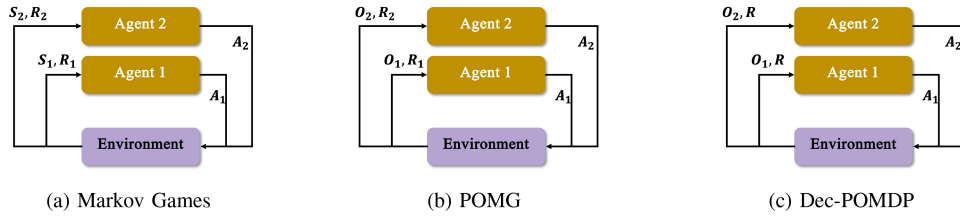


Fig. 4. Different frameworks for MARL problems.

2) *Multi-Actor Critic*: Markov games, also known as stochastic games, provide a theoretical framework for MARL by studying multiple interacting agents in a fully observable environment, applicable in cooperative, collaborative, and mixed settings. In decentralized partially observable MDPs (Dec-POMDPs), all agents aim to maximize a joint reward function, even though they have different individual goals. Partially observable Markov games (POMG), also referred to as partially observable stochastic games, correspond to Dec-POMDPs. However, instead of a joint reward function, agents optimize their respective reward functions in a partially observable environment. The comparison is illustrated in the Fig. 4.

POMG implicitly models the distribution of the belief states of other agents and is formally defined as a tuple $\langle \mathcal{I}, \mathcal{S}, \mathcal{A}_i, \mathcal{O}_i, d^0, \mathcal{P}, \mathcal{R} \rangle$, where $\mathcal{I}$ represents the set of N agents, $\mathcal{S}$ the global state set, $\mathcal{A}_i$ the action set for agent i , and $\mathcal{A} = \mathcal{A}_1 \times \mathcal{A}_2 \times \dots \times \mathcal{A}_N$ the joint action set. $\mathcal{O}_i$ defines the local observation set for agent i at global state s , while $\mathcal{O} = \mathcal{O}_1 \times \mathcal{O}_2 \times \dots \times \mathcal{O}_N$ forms the joint observation set. d^0 represents the initial state distribution, and $\mathcal{P}$ defines the state transition and observation probability set, where $\mathcal{P}(s_{t+1}, o_{t+1} | s_t, a_t)$ represents the probability of transitioning to state s_{t+1} and receiving observation o_{t+1} after taking joint action a_t in state s_t . The reward function $r_i : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ determines the reward for agent i . Within this framework, agents utilize policies $\pi_{\omega_i}(a_i | o_i)$ that are parameterized by ω_i to generate actions a_i based on their individual observations o_i . Bold font is used to denote joint quantities across all agents, with the subscript $-i$ indicating quantities for all agents except agent i (e.g., $\mathbf{a} = [a^i, \mathbf{a}^{-i}]$).

MARL algorithms typically fall into three frameworks: centralized learning, decentralized learning, and centralized training with decentralized execution (CTDE). Centralized methods learn a single policy mapping joint observations to actions [66], but face challenges with exponential growth in observation and action spaces. Decentralized learning allows agents to optimize rewards independently [89], reducing the action space from $|\mathcal{A}|^n$ to $n|\mathcal{A}|$, but may suffer from instability [62].

CTDE bridges these approaches, with methods including actor-critic structures using centralized critics [64], [73] and value-decomposition methods [68], [69], [70]. These methods have achieved state-of-the-art results in MARL benchmarks [71], [72]. MARL research spans four main areas [66], [90]: **behavior analysis**, where single-agent RL algorithms are applied to multi-agent systems; **agent communication**, which enhances learning through explicit

information exchange; **cooperative learning**, which typically uses centralized training with distributed execution to enable collaboration; and **agent modeling**, which focuses on improving cooperation by modeling other agents' strategies. Agent communication enhances performance, agent modeling addresses imperfect information, behavior analysis is straightforward to implement, and cooperative learning achieves high training efficiency. This work primarily emphasizes behavior analysis and cooperative learning due to their widespread use in multi-agent systems.

3) *Multi-Critic Multi-DQN*: Independent learning is explored in multi-agent environments through Q-learning [67], particularly in cooperative games [91]. Independent Q-learning (IQL) enables agents to learn policies individually while treating others as part of the environment [67], reducing computational requirements compared to centralized learning.

The multi-DQN framework expands DQN by incorporating IQL, where each agent i maintains its own DQN to calculate a local action-value function $Q_i(o_i, a_i)$ based on observation o_i and action a_i . This function estimates the expected cumulative future rewards for taking action a_i from state o_i under the joint policy $\pi = [\pi_i, \pi_{-i}]$. In fully observable settings, the Q-function takes the form of eq. (39),

$$Q_i^\pi(s_t, a_t^i) = E_\pi \left\{ \sum_{t=1}^{\infty} \gamma^{t-1} r_{i,t} \mid s_t = s, a_t = a_t^i \right\}. \quad (39)$$

Despite non-stationarity and uncertain convergence, IQL has shown robust practical performance [92]. The corresponding Bellman optimality for a single agent, considering other agents' policies, is expressed in eq. (40),

$$Q_i^*(s_t, a_t^i | \mathbf{a}_t^{-i}) = \sum_{\mathbf{a}_t^{-i}} \mathbf{a}_t^{-i} \left(\mathbf{a}_t^{-i} \mid s_t \right) \left[r(s_t, a_t^i, \mathbf{a}_t^{-i}) + \gamma \sum_{s_{t+1}} \mathcal{P}(s_{t+1} | s_t, a_t^i, \mathbf{a}_t^{-i}) \max_{a_{t+1}^i} Q_i^*(s_{t+1}, a_{t+1}^i) \right]. \quad (40)$$

One of the main challenges in this framework is the non-stationarity component $\pi_{-i}(\mathbf{a}_t^{-i} | s_t) = \prod_{i \in N} \pi_{-i}(a_t^{-i} | s_t)$, which changes as other agents' policies evolve during the learning process. This change creates a non-stationary environment for each agent, which goes against the Markov assumptions necessary for Q-learning convergence. The non-stationarity has a significant impact on experience replay in multi-DQN, as previous data becomes outdated and potentially misleading. This challenge can be formally

expressed as $\mathcal{P}(s_{t+1} \mid s_t, a_t, \pi_1, \dots, \pi_N) \neq \mathcal{P}(s_{t+1} \mid s_t, a_t, \tilde{\pi}_1, \dots, \tilde{\pi}_N)$.

Without replay memory, IQL can still learn by tracking other agents' policies, but replay memory hinders this process. Previous work has attempted to address the issue by inputting other agent's policy parameters to Q function [65], explicitly adding the iteration index to the replay buffer. Therefore, there are many works that extend the normal Q-function of state-action pairs $Q(s, a)$ to a function of states and joint actions of all agents $Q(s, \vec{a})$ [89], [93], [94], [95] or all other mixed strategies [65]. Off-environment importance sampling [96] can be used to stabilise experience replay [97]. Therefore, to enable importance sampling, at the time of collection t_c , $\pi_{-i}^i(a_t^{-i} \mid s_t)$ is stored in the replay memory, forming an augmented transition tuple $\langle s_t, a_t^i, r_t, \pi(a_t^{-i} \mid s_t), s_{t+1} \rangle^{(t_c)}$. At the time of replay t_r , we train off-environment by minimizing an importance weighted loss function in eq. (41),

$$\mathcal{L}(\theta_i) = \sum_{i=1}^b \frac{\pi_{-i}^{t_c}(a_t^{-i} \mid s_t)}{\pi_{-i}^{t_i}(a_t^{-i} \mid s_t)} \left[\left(y_{target}^i - Q^{\theta_i}(s_t, a_t^i) \right)^2 \right], \quad (41)$$

where t_i represents the collection time of the i -th sample, and b the batch size.

4) *Multi-Actor-Critic MADDPG*: In multi-agent RL, centralized learning causes exponential state space growth, while decentralized learning faces challenges: multi-critic methods must handle non-stationarity, and policy gradient approaches suffer increased variance as agents' rewards depend on others' actions. A key insight is that knowing all agents' actions ensures environmental stationarity, even as policies evolve, formally expressed in eq. (42), for any $\pi_i \neq \tilde{\pi}_i$,

$$\begin{aligned} \mathcal{P}(s_{t+1} \mid s_t, a_t^1, \dots, a_t^N, \pi_1, \dots, \pi_N) \\ &= \mathcal{P}(s_{t+1} \mid s_t, a_t^1, \dots, a_t^N) \\ &= \mathcal{P}(s_{t+1} \mid s_t, a_t^1, \dots, a_t^N, \tilde{\pi}_1, \dots, \tilde{\pi}_N). \end{aligned} \quad (42)$$

To address non-stationarity in IQL, hyper Q-learning [65] allows agents to learn policies based on estimation of other agents' policies, though this increases Q-function dimensionality. MADDPG [73] tackles this dimensionality issue through neural network approximation and employs a centralized critic for each agent, enabling learning of continuous policies in both cooperative and competitive scenarios. In formal terms, let us consider a game involving N agents, each with their own policies represented by the parameters $\omega = \omega_1, \dots, \omega_N$. The set of policies is denoted as $\pi = \pi_1, \dots, \pi_N$. For agent i , the gradient of the expected return $J(\omega_i) = \mathbb{E}[r_i]$ can be expressed in eq. (43),

$$\begin{aligned} \nabla_{\omega_i} J(\omega_i) \\ &= \mathbb{E}_{s \sim d^\pi, a_t^i \sim \pi_i^{\omega_i}} \left[\nabla_{\omega_i} \log \pi_i^{\omega_i}(a_t^i \mid o_t^i) Q^{\theta_i}(s_t, a_t^1, \dots, a_t^N) \right], \end{aligned} \quad (43)$$

where $Q^{\theta_i}(s_t, a_t^1, \dots, a_t^N)$ represents a centralized action-value function that takes as input the actions of all agents $a_t^1, \dots, a_t^N$ at time step t , along with state information $\mathbf{x}_t$,

and outputs the Q-value for agent i . Since each Q^{θ_i} is learned separately, agents can have arbitrary reward structures, including conflicting rewards in competitive settings. Now we can extend the stochastic policy $a_t^i \sim \pi^{\omega_i}$ to work with deterministic policies $a_t^i = \pi^{\omega_i}$ in eq. (44),

$$\begin{aligned} \nabla_{\omega_i} J(\omega_i) \\ &= \mathbb{E} \left[\nabla_{\omega_i} \pi^{\omega_i}(a_t^i \mid o_t^i) \nabla_{a_t^i} Q^{\theta_i}(s_t, a_t^1, \dots, a_t^N) \Big|_{a_t^i = \pi^{\omega_i}(o_t^i)} \right]. \end{aligned} \quad (44)$$

The experience replay buffer $\mathcal{D}$ stores tuples in the form of $(\mathbf{x}_t, \mathbf{x}_{t+1}, a_t^1, \dots, a_t^N, r_t^1, \dots, r_t^N)$, capturing the interactions of all agents. Each tuple includes the information state at time step t , $\mathbf{x}_t$, the subsequent state $\mathbf{x}_{t+1}$, the actions taken by all agents $a_t^1, \dots, a_t^N$, and the rewards received $r_t^1, \dots, r_t^N$. The centralized state-action value function Q^{θ_i} is then updated according to eq. (45),

$$\begin{aligned} \mathcal{L}(\theta_i) &= \mathbb{E}_{\mathbf{x}_t, a_t, r_t, \mathbf{x}_{t+1}} \left[\left(Q^{\theta_i}(\mathbf{x}_t, a_t^1, \dots, a_t^N) - y_t \right)^2 \right] \\ y_t &= r_t^i + \gamma Q^{\theta_i^{\text{target}}}(\mathbf{x}_{t+1}, a_{t+1}^1, \dots, a_{t+1}^N) \Big|_{a_{t+1}^j = \pi_j^{\omega_j^{\text{target}}}(o_{t+1}^j)}. \end{aligned} \quad (45)$$

The set of target policies with delayed parameters, denoted as ω^{target} , comprises of individual policies for each agent: $\pi_j^{\omega_j^{\text{target}}} = \{\pi_{\omega_1^{\text{target}}}, \dots, \pi_{\omega_N^{\text{target}}}\}$. In order to eliminate the need for prior knowledge for policies of the other agents, each agent i can also maintain an estimated policy, denoted as $\hat{\pi}_i^{\omega_j^i}$, for the actual policy π^{ω_j} of agent j . This estimated policy is refined by optimizing the log-likelihood of agent j 's action, along with an entropy regularization term as described in eq. (46),

$$\mathcal{L}(\omega_j^i) = -\mathbb{E}_{o_j, a_j} \left[\log \hat{\pi}_i^{\omega_j^i}(a_j \mid o_j) + \lambda \mathcal{H}(\hat{\pi}_i^{\omega_j^i}) \right], \quad (46)$$

where $\mathcal{H}$ represents the entropy of the policy distribution. y_t in eq. (45) can be replaced with estimated value $\hat{y}_t$ in eq. (47),

$$\begin{aligned} \hat{y}_t &= r_t^i \\ &+ \gamma Q^{\theta_i^{\text{target}}}(\mathbf{x}_{t+1}, \hat{\pi}_{\omega_1^{\text{target}}}^{\omega_1^{\text{target}}}(o_1), \dots, \hat{\pi}_{\omega_i^{\text{target}}}^{\omega_i^{\text{target}}}(o_i), \dots, \hat{\pi}_{\omega_N^{\text{target}}}^{\omega_N^{\text{target}}}(o_N)), \end{aligned} \quad (47)$$

where $\hat{\pi}_j^{\omega_j^{\text{target}}}$ represents the target network corresponding to the approximate policy $\pi_j^{\omega_j^{\text{target}}}$. To tackle high variance in competitive or cooperative environments, MADDPG introduces policy ensembles, maintaining and training K distinct policies per agent. A policy is randomly chosen for rollouts, while all K policies contribute to gradient updates. Extending DDPG with a centralized critic and decentralized actors, MADDPG enables agents to learn from estimated policies of others and reduces variance, making it effective in multi-agent settings.

5) *Multi-Actor-Critic MAPPO*: PPO has been adapted for multi-agent RL through independent PPO (IPPO) [63] and MAPPO [74]. IPPO trains separate policies for each agent, performing well in SMAC but struggling with scalability and under-performing compared to methods like QMix. MAPPO,

leveraging parameter sharing [98], allows agents to learn from collective experiences while maintaining behavioral diversity through unique observations. Its development incorporates Orthogonal initialization [99] and insights from single-agent RL [100], [101], [102], [103]. Studies suggest using larger batches and limiting training data to at most two mini-batches [102]. The actor network π^ω maps agent observations o_t to action distributions.

The actor network π^ω processes agent observations o_t to generate appropriate action distributions. The actions are then sampled from these distributions. The training objective of the network is to maximize the function defined in eq. (48),

$$L(\omega) = \mathbb{E} \left[\sum_{i=1}^n \min \left(r^{(i)} A^{(i)}, \text{clip} \left(r^{(i)}, 1 - c, 1 + c \right) A^{(i)} \right) \right] + \alpha_e \sum_{i=1}^n \mathcal{H} \left(\pi^\omega \left(o^{(i)} \right) \right), \quad (48)$$

where $r^{(i)}$ is the probability ratio between the current policy π^ω and the old policy $\pi^{\omega_{\text{old}}}$, $A^{(i)}$ and $o^{(i)}$ the actions and observations, respectively. The advantage estimation $A^{(i)}$ is calculated by GAE method. The term $\mathcal{H}$ refers to the entropy, α_e the entropy coefficient hyper-parameter. The critic network is optimized by minimizing the loss function in eq. (49),

$$L(\theta) = \sum_{i=1}^n \left(\max \left\{ \left(V_\theta \left(s^{(i)} \right) - \hat{R}_i \right)^2, \left(\text{clip} \left(V_\theta \left(s_i^{(k)} \right), V_{\theta_{\text{old}}} \left(s^{(i)} \right) - c, V_{\theta_{\text{old}}} \left(s^{(i)} \right) + c \right) - \hat{R}_i \right)^2 \right\} \right), \quad (49)$$

where $\hat{R}_i$ denotes the discounted reward-to-go, and n is the number of agents. MAPPO proposes that clipping the policy and value functions helps reduce non-stationarity from policy updates, while value normalization stabilizes value function learning.

F. Practical Techniques

One common issue in RL is the initial increase and subsequent decline of reward. To address this, it is important to first verify the code for any potential bugs. This can be done by testing the code on a benchmark, such as CartPole, to ensure proper convergence. If the code is found to be correct, the initial reward increase should be analyzed. If it matches random performance, it is possible that there may be flaws in the implementation or hyperparameters. On the other hand, if the initial reward increase is significantly higher, it could indicate that the task is too simple, leading to premature convergence. Therefore, it is crucial to ensure a well-posed MDP formulation, including the state space, action space, and reward design. Once the problem setup is sound, fine-tuning the learning rate, entropy coefficient, and batch size can further improve performance.

In order to ensure successful convergence, it is essential to normalize the state, particularly when using clipping mechanisms like PPO. Action spaces should also be scaled within a range of $[0, 1]$ or $[-1, 1]$, with any necessary rescaling being handled within the environment. Additionally, providing

manually computable features can help reduce the learning burden on the agent. While SAC is efficient with limited data, PPO is better suited for large-scale settings and is generally the most reliable choice. It should also be noted that transformer-based models require a significant amount of data for effective training.

The applicability of DRL to a communication problem depends on whether it can be effectively modeled as an MDP. If environmental changes follow a consistent distribution, they can be incorporated within the MDP framework. However, if these changes alter the MDP's fundamental properties, leading to a shift to a different MDP, it is essential to reassess the accuracy of the modeling. In such cases, alternative approaches like continual learning or modifications to DRL (e.g., specific training strategies or architectural adjustments) may be required for effective problem-solving.

III. VALUE-BASED DRL FOR POWER ALLOCATION OPTIMIZATION

We investigate DRL applications in the field of power allocation into two categories, one-hop [104], [105], [106], [107] and multi-hop [108], [109], [110], [111] scenarios to highlight the distinct challenges and approaches associated with each type of network topology. In one-hop scenarios, communication occurs directly between a UAV and its base station or other communication nodes. The optimization problem typically focuses on maximizing power efficiency, reducing interference, and ensuring reliable communication while minimizing energy consumption. In this scenario, the problem is more straightforward, as the optimization is localized to the interaction between two nodes. On the other hand, multi-hop scenarios involve communication across multiple UAVs or between a UAV and several relay stations, creating a more complex network topology. Here, power allocation must consider the cooperative interaction between several UAVs, accounting for the energy consumption of intermediate nodes, interference from neighboring nodes, and the need for maintaining signal strength across extended distances. The optimization becomes more complex due to the multi-agent nature of the problem, requiring coordination and distributed decision-making. The summary of DRL implementation for problem formulations of power allocation is provided in Table III.

A. Value-Based DRL in One-Hop Scenarios

In the one-hop scenario, [104], [105], [106] adopt the DQN algorithm, while [107] applies the DDQN algorithm. Traditional resource allocation methods face challenges in ensuring stable communication coverage due to the high mobility of UAVs and the instability of air-to-ground channels [104]. Fig. 5 depicts the scenario analyzed in [104], which involves UAV 1 and users 1 and 2. To ensure the continuity of communication coverage between UAV 1 and users 1 and 2, the channel allocation and power allocation of UAV 1 are optimized. The paper proposes a joint allocation of sub-channels and power to meet the coverage requirements of ground users, employing a DQN algorithm to solve this

TABLE III
POWER ALLOCATION

Scenario	Work	Alg.	Rewards	Action Constraints		
				Direct Range	0-1 Combination	Others
One-hop	[104]	DQN	Inconsistent	NM	NM	NM
	[105]	DQN	Consistent	NM	NM	NM
	[106]	DQN	Consistent	PR	/	NM
	[107]	DDQN	Consistent	PN, NS	/	NM
Multi-hop	[108]	DQN	Consistent	/	NM	PR
	[109]	DDQN	Inconsistent	PR	NM	NM
	[110]	DQN	Inconsistent	NM	/	NM
	[111]	DDQN	Consistent	NM	NM	NM

Inconsistent/Consistent: The reward is inconsistent/consistent with the objective in the problem formulation. Direct Range: There is a constraint on ranges for actions. 0-1 Combination: The actions are binary variables, and their linear sum remains a binary result. PD: Direct Penalty; PR: Penalty in Rewards; NS: Network Structure; NM: Not mentioned how to implement constraints in DRL proposals.

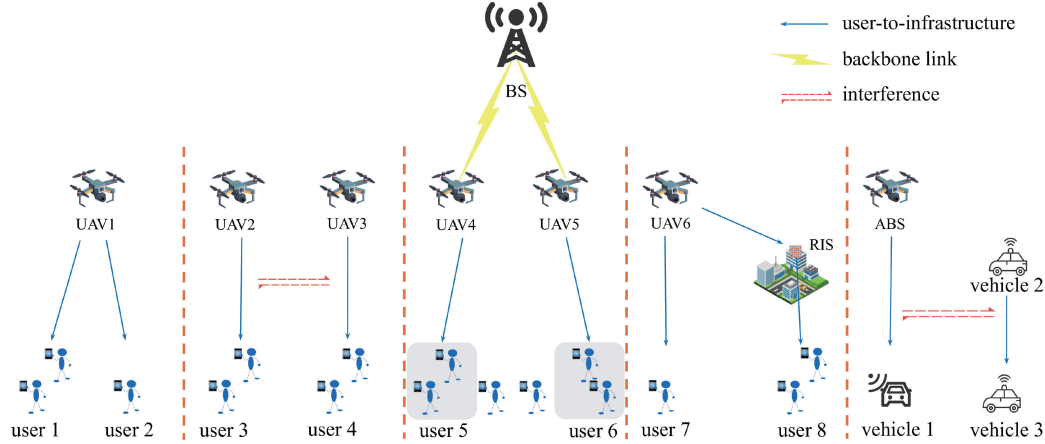


Fig. 5. Power allocation in different scenarios.

problem. The problem formulation aims to maximize spectral efficiency and minimize rate variance at each time slot, while the DRL framework focuses on maximizing cumulative rewards over all time slots. As a result, the solution derived from the DRL algorithm is not entirely aligned with the original problem's objective.

The study does not detail methods for incorporating constraints within the DQN framework, but several strategies can be proposed. For the transmit power range constraint, a ReLU activation function can ensure non-negative outputs. To enforce the sum of transmit power constraint, a penalty term can be added to the reward function, where exceeding the power threshold incurs a negative reward. Sub-channel allocation can be treated as a binary classification problem, with a sigmoid activation function used in the output layer to restrict values to $[0, 1]$, which is then rounded to binary outputs. Similarly, the connection indication symbol constraint can be implemented by applying a penalty-based approach to the reward function, ensuring compliance while guiding the agent's learning process. These methods enhance the DQN framework's ability to address multiple constraints effectively.

A similar study to [104] explores the optimization of bandwidth allocation, transmission power, and UAV scheduling in [105]. The mathematical model in this work aims to maximize the long-term average total data transmission, while its DQN-based approach seeks to maximize cumulative rewards. The solution obtained through the DQN algorithm aligns with the objective of the problem formulation, with the

reward function defined as the amount of data transmitted. However, the paper does not detail methods for implementing the constraints outlined in the formulation model. To address the constraint on the cluster service indicator for each UAV, the constraint on the total bandwidth, and the constraint on power range, penalty terms can be added to the reward function. If a constraint is violated, the corresponding penalty term is set to a negative value to reduce the reward; otherwise, it is set to zero. For the constraint, where the cluster service indicator is binary (0 or 1), this can be enforced through the network structure by using a softmax function to generate a probability distribution over possible values, followed by one-hot encoding to set the highest-probability item to 1 and others to 0. The constraint related to battery capacity, and the final constraint concerning the energy of sensor nodes, can be implemented by introducing coefficients for the respective variables. The actual value of each variable is calculated as the product of its upper bound and the coefficient, which is constrained to the range $[0, 1]$ using a sigmoid activation function. These methods provide a systematic way to incorporate the constraints into the DQN framework.

Additionally, trajectory design and power allocation in [106] are jointly coupled to enhance spectral efficiency and optimize system throughput while employing non-orthogonal multiple access (NOMA) in UAV-assisted cellular networks. As depicted in Fig. 5, this study involves UAVs 2 and 3, along with users 3 and 4. By jointly optimizing the trajectories and power allocation of UAVs 2 and 3, it effectively minimizes

channel interference between the UAVs, thereby enhancing overall throughput. The objective in the formulation model is to maximize system throughput across multiple time slots, aligning with the DQN goal of maximizing cumulative rewards. The solution derived from the DRL algorithm matches the optimal solution of the model's objective. To implement the sixth constraint, a penalty term is incorporated into the reward function as part of the denominator. When the constraint on the data transmission rate is violated, the penalty term takes a large value, significantly reducing the reward and guiding the agent to adhere to the constraint.

However, the study does not detail the implementation methods for the remaining constraints. These constraints can be addressed by adding penalty terms to the reward function. If a constraint is violated, the penalty term takes a negative value; otherwise, it remains zero, leaving the reward unaffected. Alternatively, a direct penalty approach can be adopted, where any constraint violation replaces the reward with a negative penalty value. For the constraint regarding UAV trajectories, adjustments to the DQN network structure can be made. For example, a height coefficient can be introduced for UAV altitude, with the actual altitude calculated as the product of the maximum allowable height and the coefficient. The height coefficient is constrained to the range $[0, 1]$ using a sigmoid activation function, ensuring non-negative altitudes. A similar approach can be applied to the horizontal and vertical trajectory coordinates of UAVs, using a tanh function to map the coefficients to the range $[-1, 1]$.

Finally, energy consumption is another consideration in UAV networks. [107] addresses this challenge by jointly optimizing content placement, spectrum allocation, co-channel link pairing, and power control to enhance UAV energy efficiency while ensuring user quality of service (QoS). As shown in Fig. 5, the study involves an aerial base station (ABS) and vehicles 1, 2, and 3. By optimizing power allocation between the ABS and vehicles 1 and 2, as well as between vehicle 3 and vehicle 2, the energy efficiency of the ABS is significantly enhanced. Its objective in the problem formulation is to maximize the average energy efficiency over multiple time slots, while the goal of DRL is to maximize cumulative rewards. The solution obtained by its DDQN algorithm is consistent with the solution for the formulation objective. Additionally, to implement the first constraint on the range of the transmission rate in its DDQN algorithm, a penalty term is added to the reward function. When the transmission rate fails to meet the first constraint, the penalty term takes a negative value; otherwise, the penalty term is 0. To implement the fourth and fifth constraints on the range of the transmission power in its DDQN algorithms, the actions are discretized and their range is confined within the bounds specified by the constraints.

However, the proposal does not outline specific methods for implementing the remaining constraints within the DRL framework. To implement the constraint on the reliability requirement and the constraint on the average delay, we can follow the penalty approach in its DDQN algorithm. Specifically, a penalty term is added to the reward function.

B. Value-Based DRL in Multi-Hop Scenarios

In multi-hop scenarios, typical problems, such as power allocation, spectral assignment, and UAV trajectory optimization [108], [110], can benefit from the application of DRL algorithms to enhance network performance. Multi-hop communication inherently involves larger complexity due to the need to manage interactions across multiple nodes, making DRL's ability to handle high-dimensional state and action spaces particularly advantageous. Moreover, multi-hop networks support a variety of applications, such as video transmission [109] and traffic forwarding [111], which require efficient resource management and low-latency communication. DRL-based approaches can dynamically allocate resources and adapt to changing network conditions, ensuring reliable data delivery and optimal performance in these scenarios.

Typical Issues in Multi-hop Scenarios: To handle communication disruptions caused by infrastructure damage in disaster scenarios [108], UAVs can be deployed as information relays between the macro base station (BS) and users, helping to restore communication links. As shown in Fig. 5, the key elements in this example include the BS, UAVs 4 and 5, and users 5 and 6. The BS achieves communication with users 5 and 6 by treating UAVs 4 and 5 as relay nodes. The overall spectral efficiency is improved by optimizing the transmission power allocation of the BS and the service area allocation of UAVs 4 and 5. The objective of the problem formulation is to maximize the average spectral efficiency, while the DQN algorithm focuses on maximizing cumulative rewards. By using spectral efficiency as the reward function in the DQN algorithm, the solution produced by the algorithm aligns with the original problem objective. To implement the constraint, which limits the total allocated power, a direct penalty method is applied. Specifically, if the constraint is violated, the reward is set to a negative value. Otherwise, the reward remains non-negative.

However, the methods for implementing the constraints on the zone selection are not mentioned in the paper. The constraints are about the 0-1 variable, which can be implemented by configuring the network structure accordingly. Specifically, a softmax activation function is used to transform all outputs into a probability distribution, ensuring the probability sum to 1. The action with the highest probability is then selected as the output action.

Reconfigurable Intelligent Surface (RIS) is a key component in 6G and future networks. When integrated with UAV networks, the complexity of the optimization increases, particularly when jointly optimizing UAV trajectories, power allocation, and RIS phase shifts [109], [112]. For instance, in Fig. 5, the elements considered include the RIS, UAVs 2, 3, and 6, along with users 3, 4, 7, and 8. When there are obstacles preventing direct communication between UAV 6 and users 7 and 8, the RIS can act as a relay node to enable communication between UAV 6 and users 7 and 8. By jointly optimizing the movement trajectories and power allocation of UAVs 2, 3, and 6, as well as the RIS phase shifts, signal interference between UAVs is minimized and the overall throughput is improved.

Its objective of the problem formulation is to maximize the system throughput in each time slot, while the DRL algorithm is designed to maximize cumulative rewards over all time slots. This creates an inconsistency between the problem objective and the solution obtained by its DRL algorithm. Additionally, to enforce the constraints on the UAV's flying area and the equivalent channel coefficient, penalty terms are added to the reward function. If any of these constraints are violated, a significant penalty is subtracted from the reward to discourage such actions and guide the learning process toward feasible solutions.

However, the implementation methods for the remaining constraints are not explicitly discussed in the paper. The constraint on the feasible RIS phase shift range and the constraint on the total UAV energy consumption can be easily addressed within the DRL algorithm. These constraints are straightforward to incorporate as they involve simple boundary checks or adjustments to ensure compliance. For the constraint involving the service indication mark between users and UAVs, it is recommended to follow the approach outlined in the paper. This involves introducing penalty values into the reward function. By assigning different penalty values based on the specific violations of various constraints, the DRL algorithm can effectively discourage actions that violate the constraint and encourage feasible solutions. Furthermore, for constraints directly related to DRL actions, such as the constraint to prevent UAV collisions and the constraint on the total allocated power, the action mask approach is an effective solution. In this approach, action options that violate the relevant constraints are filtered out before an action is selected. This ensures that the DRL agent operates within a valid action space, avoiding constraint violations and improving both the reliability and efficiency of the learning process.

Multi-hop Applications: Reference [110] investigates the challenge of efficient video transmission in UAV-assisted wireless networks by jointly optimizing video quality level selection and power allocation. The objective is to maximize the ratio of video quality to power consumption, thereby improving energy efficiency while ensuring secure video transmission. While the problem formulation aims to maximize energy efficiency at each time slot, the DRL algorithm is designed to optimize cumulative rewards across all time slots, leading to a potential misalignment with the formulation objective.

However, the article does not detail the implementation of constraints. For the constraint regarding the secrecy time-out probability, a penalty-based approach can be adopted by incorporating a penalty term into the reward function. Violations of this constraint result in a significant reduction in the reward, guiding the DRL agent toward actions that satisfy the constraint. The constraint regarding selecting suitable layers from a group of pictures encoded from videos, can be addressed by structuring the output layer of the DRL action network such that each neuron corresponds to a possible action. Applying a softmax activation function to this layer transforms the outputs into a probability distribution, and the action with the highest probability is selected. To handle the constraints concerning the power allocation range, the

action variables can be modified using an action coefficient mechanism. This involves scaling each action variable by its corresponding action coefficient, where the true action value is the product of the coefficient and the maximum possible value. A sigmoid activation function ensures that these coefficients remain within the given range, thereby enforcing compliance with the power allocation constraints. By adopting these methods, the DRL algorithm can effectively integrate the constraints into its optimization process, ensuring improved video transmission efficiency and security.

When UAVs are integrated with cognitive radio technology, energy efficiency of traffic forwarding becomes a critical concern. Reference [111] addresses this issue by exploring the use of cognitive radio technology-enhanced UAVs (CUAVs) to assist in data traffic forwarding within IoT networks. In this configuration, UAVs act as mobile relay stations, collecting data from ground-based IoT devices and forwarding it to BSs. By exploiting idle spectrum resources, the UAV establishes wireless backhaul links, enabling efficient data transmission.

The objective of this paper is to maximize energy efficiency over multiple time slots, while the goal of the DRL algorithm is to maximize cumulative rewards. The solution obtained through the DRL algorithm aligns with the target solution of the problem, as both aim to optimize energy efficiency in a long time. To apply the DRL algorithm, the reward is set as energy efficiency, allowing the agent to learn optimal policies that balance energy consumption for network performance.

However, the solution does not provide explicit methods for implementing the constraints. To address this, we propose the following solutions for each constraint. For the constraints that pertain to the service area selection for CUAVs, we can implement them by defining the network structure. Specifically, a softmax function can be used to generate a probability distribution over all possible values of the decision variable. One-hot encoding can be applied to select the value with the highest probability to 1, and the remaining values can be set to 0. The action corresponding to the selected value of 1 is then chosen as the output action, ensuring the selection of valid service areas for the CUAVs. For the constraints for spectral selection, and the constraint on the remaining energy, penalty terms can be incorporated into the reward function. If any of these constraints are violated, the corresponding penalty is added to the reward. The penalty reduces the reward value, discouraging the agent from selecting actions that lead to violations of these constraints. To implement the constraint, which restricts the power allocation range, a power coefficient can be introduced. The actual power value is determined by multiplying the maximum power limit by the power coefficient. This transforms the power allocation problem into setting the power coefficient, and a sigmoid activation function can be used to constrain the power coefficient to the range $[0, 1]$, ensuring that the power allocation is within the specified bounds. For the constraint involving limiting certain decision variables to the range $[0, 1]$, we can directly use a sigmoid activation function to restrict the variable to the desired range, ensuring the constraints are respected during the DRL process.

TABLE IV
CHANNEL ASSIGNMENT

Scenario	Work	Alg.	Rewards	Action Constraints		
				Direct Range	0-1 Combination	Others
One-hop	[113]	DDPG	Consistent	PR	/	NS
	[114]	DDPG	Consistent	NM	NS	/
	[115]	DDPG	Consistent	/	NM	NM
	[116]	DDPG	Consistent	NS	NM	NM
	[111]	DDPG	Consistent	NM	NM	NM
Multi-hop	[117]	TRPO	Consistent	PD,NS	NM	NM
	[118]	PPO	Consistent	PR,NS	/	/
	[119]	PPO	Inconsistent	NM	NM	PD

IV. POLICY-BASED DRL FOR CHANNEL ASSIGNMENT OPTIMIZATION

We explore the application of policy-based DRL techniques to optimize channel assignment in UAV communication networks. This section is divided into two subsections: DRL in one-hop scenarios and DRL in multi-hop scenarios, each addressing the unique challenges and solutions for channel assignment in different communication settings. We focus on problems in the former where UAVs communicate directly with ground users or base stations using a single-hop communication model. The objective is to efficiently allocate communication channels between the UAV and users, ensuring that network performance is optimized in terms of throughput, fairness, and interference mitigation. We explore how policy-based DRL can be applied to dynamically assign channels, adapting to changes in the environment, and making decisions that maximize system overall efficiency. The communication is more complex in the later, involving multiple UAVs and users, where data is forwarded across several hops before reaching its destination. This scenario presents additional challenges, such as managing inter-UAV interference, coordinating the channel assignment among multiple UAVs, and handling the dynamic topology of the network. We investigate how policy-based DRL can be used to optimize channel assignment in multi-hop networks, focusing on improving network throughput, minimizing delays, and ensuring efficient resource utilization in more complex communication environments. The summary of DRL implementation for formulations of channel assignment problem is listed in Table IV.

A. Policy-Based DRL in One-Hop Scenarios

We explore two key categories of research focusing on the DRL application in optimizing channel assignment in UAV communication networks under one-hop scenarios. The first category addresses typical issues encountered in one-hop communication settings, including energy efficiency, fairness, and channel interference. These issues are crucial for ensuring efficient and reliable communication between a UAV and ground users or base stations. DRL techniques are applied to optimize channel assignment, reduce power consumption, balance fairness among users, and mitigate interference, thereby enhancing system throughput and overall network performance [113], [114], [115]. The second category focuses on one-hop applications within UAV networks, such as data forwarding and task offloading [111], [116]. Although these

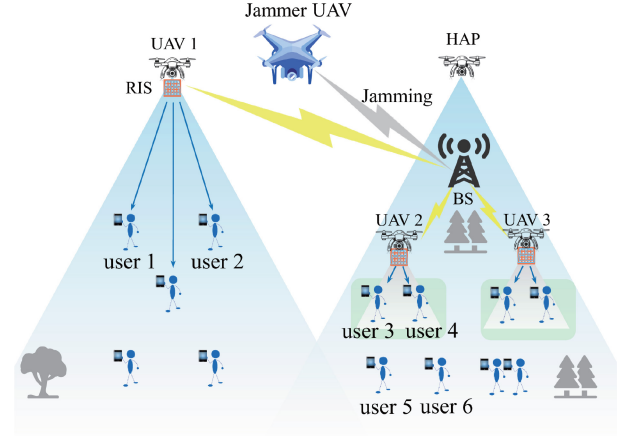


Fig. 6. Channel assignment in different scenarios.

applications involve one-hop communication, they present unique challenges like QoS requirements for applications. DRL can be leveraged to dynamically manage UAV resources, optimize the forwarding of data to ground stations or users, and determine efficient offloading strategies for computational tasks.

Typical Issues in One-hop Scenarios: The issue of balancing energy efficiency and fairness is addressed in UAV-assisted communication systems [113]. The main problem arises from the fact that a UAV, while optimizing its communication performance, also maintains fairness in resource allocation to all users over time. In Fig. 6, UAV 1 provides communication services for users 1 and 2. To address these challenges, the study proposes a DDPG-based approach that aims to optimize the 3-D trajectory design and channel assignment for the UAV. The DDPG reward in the paper captures the trade-offs involved in optimizing both fairness and energy efficiency, with the UAV adjusting its trajectory and resource allocation accordingly. Furthermore, the study introduces the concept of penalty rewards to implement various constraints within the DRL framework. These penalty rewards are designed to ensure that all operational constraints, such as UAV acceleration limits, are respected throughout the optimization process, ensuring that the UAV acceleration stays within specified bounds. Similar penalty terms are used for other constraints, such as those related to power allocation and communication quality.

In the design of the reward function, the study does not clearly explain how some parameters, like distance metrics

and UAV coordinates, are updated during the optimization process. Specifically, there is no detailed description of how the UAV speed and directional components in the action space affect its position in the state space. Importantly, the paper provides comprehensive guidance on implementing constraints effectively within the DDPG framework, ensuring robust compliance with all system requirements.

The fairness and network throughput influenced by UAV mobility and channel assignment, similar to the work [113], are considered in [114], but without information of user locations and channel parameters. Taking UAVs 2 and 3, and users at left in Fig. 6 as an example, optimizing channel assignment for the two UAVs can mitigate channel interference while multiple orthogonal channels are shared. The authors propose a DDPG framework that incorporates a fairness indicator to optimize throughput while ensuring fairness in resource distribution. The objective in the problem formulation is to maximize both the overall throughput and the fairness across consecutive time slots, which is consistent with the reward in the DDPG proposal. Regarding the constraints, the paper focuses on converting discrete vector actions into a decimal integer, addressing specific operational limitations of the UAVs in terms of action selection.

However, some constraints in the problem formulation are not fully addressed. One significant physical limitation is the restriction on UAV movements to ensure they do not exceed the maximum allowable distance during their trajectories. This can be done by normalizing the movement vector based on the maximum allowable distance. Another important constraint concerns power allocation, specifically ensuring that the total power across all channels does not exceed the maximum power capacity available to the system. To implement this constraint in the DDPG proposal, a power allocation sub-network can be integrated into the policy framework. The sub-network outputs power values that are then processed by a normalization layer.

Policy-based DRL for channel assignment is also studied in [115] with a similar scenario to [114]. The main difference lies in two assumptions. The first is that users are grouped into clusters. The second is that the cluster movement and UAV trajectories are described by a given model. Both of them are determined prior to channel assignment. The approach jointly optimizes channel assignment, power control, and user association to enhance cumulative throughput, demonstrating superior performance and effective coverage through simulations. The objective in the problem formulation is to maximize the cumulative throughput while ensuring uninterrupted services and meeting QoS requirements across multiple time slots. Meanwhile, the goal of the proposed DDPG scheme is to maximize cumulative network throughput.

However, the work [115] does not explicitly detail how constraints are implemented within the DRL framework. Therefore, we propose methods to incorporate these constraints effectively. The first constraint is related to the minimum height separation between UAVs to prevent collisions. To implement the constraint in the DRL framework, a penalty mechanism can be introduced: when this constraint is violated, a significant negative reward or penalty is applied. It discourages the agent from selecting actions that lead to

violations. Another constraint defines the binary variable for channel assignment and user association. DDPG can include a mechanism where selecting invalid combinations results in a penalty or reduced reward. Furthermore, the constraint ensuring that each subcarrier is assigned to only one mobile user at a given time can be implemented in DRL by validating actions before execution. Penalties are applied when overlapping assignments are detected. For the constraint on the maximum transmit power of the UAVs, this can be integrated into DRL by monitoring power allocation during actions and imposing a negative reward when the power exceeds the given threshold. This discourages the agent from over-allocating power. Finally, a constraint requires that the SINR between the UAV and mobile users meets a minimum threshold. The reward function can be designed to include SINR checks: if an action results in an SINR below the threshold, a penalty is assigned. In summary, these methods ensure that all constraints are effectively implemented in the DRL framework.

One-hop Applications: UAVs and high-altitude platforms (HAPs) can function as computing servers, allowing ground users with limited resources to offload tasks for processing [116]. For instance, as shown in Fig. 6, users sharing the channel resources can offload their tasks to the HAP or UAVs 2 or 3 for processing. The primary goal in the problem formulation is to maximize the number of IoT devices whose tasks are completed within deadline constraints while minimizing overall energy consumption. The goal in the problem formulation focuses on long-term optimization, which aligns with the reward design in its DDPG proposal.

Although the study outlines the implementation of a specific constraint within the DRL framework, it overlooks some additional constraints that are essential for optimal performance. To implement the constraint related to task association, one solution is to use action discretization techniques. This approach converts the continuous outputs into discrete actions, ensuring that only feasible associations are considered when assigning IoT devices to UAVs or HAP. For the constraints on the total bandwidth allocation within a given threshold, one approach is to use a projection operator. The operator adjusts the bandwidth allocation at each decision step to ensure that the selected bandwidth stays within the allowable threshold. Additionally, the constraint related to energy consumption can be managed by integrating energy efficiency directly into the reward function. This integration encourages the model to prioritize energy savings while still meeting the other performance goals, such as completing tasks on time.

One assumption in [116] is that UAVs follow their predefined trajectories based on the pre-programmed flight plans. System performance can be improved by designing UAV trajectories. The study [111] proposes a joint design strategy focusing on channel availability and UAV trajectories while taking into account traffic demands and energy refreshment over time slots in UAV-assisted cognitive radio IoT networks. The DRL-based solution aims to maximize cumulative rewards, which implements the formulation objective.

The proposal does not provide a clear explanation to implement the constraints within the DRL framework. For example, the proposal imposes a constraint to ensure that a UAV only

selects one serving area at a time. This can be implemented in the DRL framework by generating a probability vector and selecting the action with the highest probability. Another constraint involves ensuring that a UAV accesses a specific number of sub-channels from the available pool. This can be achieved in the DRL framework by applying a mask during action selection. Additionally, the paper includes a constraint requiring that the remaining energy of a UAV remains above a threshold. This can be implemented by incorporating a penalty into the reward function. Further, there is a constraint on the transmission power of UAVs. This can be enforced in the DRL framework by using clipping techniques to ensure that the chosen transmission power stays within bounds. Another constraint ensures that some variables remain within a defined range, which can be achieved through normalization or bounded activation functions. Lastly, constraints related to binary decision variables for area selection and channel access can be implemented in the DRL framework by incorporating a softmax layer and using binarization logic during the post-processing of the action vector to ensure binary outputs.

B. Policy-Based DRL in Multi-Hop Scenarios

In this subsection, we also categorize studies into two groups: typical issues and multi-hop applications. The first category addresses typical issues that arise in multi-hop scenarios, such as channel interference and trajectory design. The second category focuses on multi-hop applications while considering channel assignment, which are more complex and involve advanced network functionalities. Examples include channel jamming and age of information (AoI) in industrial IoT. Channel jamming presents a significant challenge in multi-hop networks, where malicious interference from a mobile UAV can degrade communication performance. On the other hand, UAVs often assist in data collection and transmission, and managing the age of information is crucial to ensure timely data delivery while delays are influenced by channel assignment. This subsection demonstrates how policy-based DRL can be effectively applied to solve these issues.

Typical Issues in Multi-hop Scenarios: Both energy efficiency and collision avoidance in UAV networks are closely linked to channel assignment and trajectory design [117], especially when UAVs make decisions in a distributed manner. In Fig. 6, the network consists of UAVs 2 and 3, users 3 and 4. The objective in its problem formulation is to maximize energy efficiency by optimizing UAV trajectories and channel allocation across multiple time slots in a THz-enabled UAV network, which is aligned with the reward in its DRL framework. The work specifically implements collision-related constraints through direct penalties and adjustments to the network structure. However, the paper does not fully address how to implement the constraint related to the minimum QoS for IoT devices in the DRL framework. To address the minimum QoS levels, we introduce a dual-component reward function that encourages the agent to meet requirements from IoT devices, penalizing deviations from thresholds.

Multi-hop Applications: Jamming threats are mitigated by considering the impact of channel assignment within a local area in hybrid RIS-mounted UAV networks [118]. For example, UAV 1, by utilizing RIS, enhances the anti-jamming performance for communications between the BS and users in Fig. 6. The objective of the problem is to maximize long-term energy efficiency by jointly optimizing channel assignment for the RIS system, which is achieved through the reward design in its PPO-based approach. There are constraints on transmission power, channel assignment, and SINR requirements, which are incorporated into the PPO-based approach through a single penalty in the reward function. While this approach theoretically works, it has some practical drawbacks, as the impact of these constraints on energy efficiency varies. Moreover, UAV mobility is not considered in the model. For example, adjusting UAV trajectories closer to the receiver can help mitigate jamming threats.

Hybrid RIS-mounted UAV networks are also considered in order to manage AoI in industrial IoT [119], [120]. The network in [119] consists of the BS, HAP, UAVs 2 and 3, and users in Fig. 6. IoT devices can transmit data directly to the BS or via reflection through RIS-mounted UAVs. The channel assignment for links between IoT devices and the BS, along with UAV trajectories and RIS configuration, significantly impacts the objective of minimizing data AoI. The main objective in its problem formulation is to minimize the maximum AoI across all devices over time, which is closely aligned with the long-term cumulative reward in its PPO algorithm.

In terms of implementing its constraints within the DRL framework, the paper primarily addresses one constraint, where violations lead to the termination of learning. However, for the implementation of the other constraints, several methods can be applied. For UAV trajectories, constraints are introduced to ensure that UAV movement stays within predefined areas. This can be achieved by rescaling or clipping the output of the policy network to enforce boundary conditions on the UAV's paths. The proposal can implement power constraints by normalizing the power allocation variables to ensure that they stay within the prescribed bounds. Similarly, to ensure proper power scaling for each user, the power output is scaled down proportionally to meet the required constraints. For binary decisions, such as subcarrier assignment, the system can employ a binary encoding technique. The policy network outputs an integer within a defined range, which is then mapped to a binary vector. Each bit of this vector represents the state (0 or 1) of the corresponding element in the subcarrier assignment. To ensure that RIS phase shift stays within the allowable range, a nonlinear activation function like tanh can be applied, and it is scaled and shifted accordingly. Likewise, to keep the reflection amplitude within the desired range, a sigmoid function can be used to enforce the amplitude constraint.

Besides, it is worth noting that data can be relayed among multiple UAVs in this scenario. However, the study primarily focuses on how to channel assignment and configure RIS settings, without addressing the design of routing paths for data transmission.

TABLE V
CACHING

Scenario	Work	Alg.	Rewards	Action Constraints		
				Direct Range	0-1 Combination	Others
Caching without UAV Interaction	[121]	TD3	Consistent	PR	NM	PR
	[122]	TD3	Inconsistent	NM	NM	NM
	[123]	PPO	Consistent	NM	NM	NM
	[124]	PPO	Consistent	NS	NS	NM
	[125]	PPO	Consistent	NM	NM	NM
Caching with UAV Interaction	[126]	PPO	Consistent	/	NM	NM
	[127]	DDPG	Consistent	PR	PR	PR
	[128]	DDPG	Inconsistent	NM	NM	NM

V. ACTOR-CRITIC BASED DRL FOR CACHING OPTIMIZATION

We investigate DRL applications in the field of caching into two categories, caching without UAV interaction and caching with UAV interaction scenarios. This classification is based on whether UAVs operate independently or collaborate to optimize caching strategies. In caching without UAV interaction, individual UAVs focus on optimizing their own caching decisions to enhance content delivery, without explicit coordination with other UAVs, as explored in [121], [122], [123], [124], [125]. On the other hand, caching with UAV interaction involves multiple UAVs collaborating to share cached content, adjust their trajectories, and manage resources collectively, as discussed in [126], [127], [128]. This separation highlights the differing complexities and performance benefits associated with independent and cooperative UAV caching strategies. The summary of DRL implementation for problem formulations of caching optimization is provided in Table V.

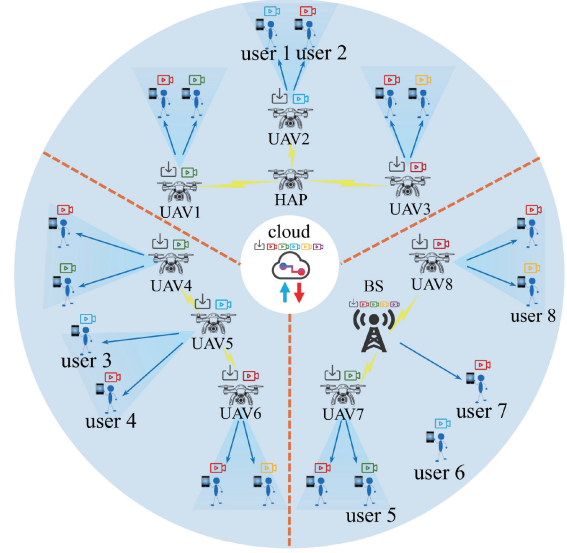


Fig. 7. Caching optimization in different scenarios.

A. Caching Without UAV Interaction

In the caching without UAV interaction scenarios, [121] and [122] apply the TD3 algorithm, while [123], [124], and [125] utilize the PPO algorithm. These studies explore different approaches to optimize caching decisions in wireless networks, with each algorithm offering unique benefits in terms of stability, convergence, and learning efficiency.

TD3 based Caching: In the Internet of Vehicles (IoV) environment, mobile edge computing (MEC) systems face challenges in ensuring QoS and efficiently managing data caching. Reference [121] proposes a joint UAV-assisted IoV architecture that leverages the mobile flexibility and line-of-sight communication capabilities of UAVs, combined with DRL methods, to optimize data caching and computation offloading, and reduce task processing delays. As illustrated in Fig. 7, UAV 8, users (vehicles) 7 and 8, and the BS are involved in the scenario. User 8, located outside the service area of the BS, can benefit from QoS by accessing cached information provided by UAV 8. The objective in its problem formulation is to maximize UAV cache hit ratio and minimize average task processing delay, which is consistent with the reward in its TD3 algorithm. In addition, when the environmental constraints on cache capacity and processing delay are not met, a large negative penalty is added to the reward function to guide the agent towards feasible solutions.

However, the paper does not specify methods for implementing resource allocation and binary decision constraints. Below are several approaches to implement these constraints in the DRL framework. For the resource allocation constraint, continuous action spaces can be used, where the action values represent decisions for resource allocation. The output can be constrained within the range of $[0, 1]$ using clipping or activation functions such as sigmoid or tanh. Additionally, a penalty term can be added to the reward function, which applies a negative reward if the total resource allocation exceeds the allowed threshold. For the binary decision constraint, a common method is to define the action space as binary, where each action (such as task offloading or data caching) can only take two values: 0 (no action) or 1 (action taken). This ensures that decisions regarding task offloading, data caching, or other discrete actions are binary, as required by the problem formulation. The binary decisions can be implemented using techniques such as the Gumbel-softmax trick, which allows continuous relaxation of binary decisions, enabling the DRL algorithm to optimize these choices efficiently.

The UAV-assisted dynamic data caching and computation offloading strategy proposed in [121] aims to minimize task processing delays and energy consumption, thus enhancing system performance. However, in multi-UAV-assisted edge computing systems, the optimization of service caching and

task offloading becomes even more challenging due to network dynamics, storage and energy restrictions. To overcome this, [122] proposes a joint UAV-based architecture that integrates task popularity with a greedy algorithm. As illustrated in Fig. 7, UAVs 7 and 8, users, and the BS are involved in the scenario. The request delay of the users benefits from the cached data of UAVs 7 and 8, while the BS handles the small amount of data not cached by the UAVs. The problem objective minimizes the overall system delay in each time slot, while its DRL algorithm aims to maximize cumulative rewards. The solution obtained by PPO differs from that in each time slot.

However, the paper does not specify exact methods in its PPO framework for implementing the constraints. To implement energy consumption, resource capacity, and geographic coverage constraints, a general approach is to use the penalty method. In this method, a penalty term is added to the reward function in DRL to discourage constraint violations. For example, to ensure that energy consumption limits are respected, if the energy consumed by a device or UAV exceeds its specified maximum energy budget, a penalty is applied to the reward. This penalty reduces the agent reward, guiding it to avoid actions that violate the energy constraints. For binary constraints, such as offloading decisions and data caching, the sigmoid activation function can be applied to constrain the output within the range of $[0, 1]$. A threshold (e.g., 0.5) can then be set, where outputs greater than 0.5 are set to 1 (action taken), and outputs less than or equal to 0.5 are set to 0 (no action). This binary decision-making approach ensures that the DRL algorithm only selects discrete actions while adhering to the problem constraints. To handle continuous constraints, such as resource allocation and task offloading, the sigmoid function can also be applied. The sigmoid function maps continuous variables to the range of $[0, 1]$, making it suitable for normalizing and restricting values within the acceptable limits for continuous decision variables.

PPO based Caching: Although the service caching optimization strategy proposed in [122] provides significant improvements, the challenge of dealing with uncertainties in user requests and network dynamics still remains, especially in multi-access environments. To address these issues, [123] integrates edge caching and UAV communication by incorporating UAV trajectory optimization. It reduces content acquisition delays and enhances the performance of UAV systems in highly dynamic environments, by means of PPO to adapt to the unpredictable network states. The formulation objective is to minimize the total content retrieval delay for all users in the cell over the entire time period. The solution obtained by the PPO algorithm aligns with the objective. To apply DRL and encourage agent exploration, the reward consists of both extrinsic and intrinsic components, where the objective function influences the extrinsic reward, and the agent's exploration behavior affects the intrinsic reward.

However, the paper does not specify the exact methods for implementing the constraints in its PPO framework. Below are several approaches for implementing these constraints. For the binary decision variable constraints related to cache capacity, cache diversity, and user association, penalty terms can be

added to the reward function for violations. For the constraints concerning transmission power and UAV movement, the PPO output values are mapped using sigmoid or tanh functions to ensure they remain within the corresponding physical constraint ranges.

While UAV trajectory optimization has been shown to effectively reduce content acquisition delays [123], the need for real-time updates to the caching strategy remains critical for addressing the dynamic and unpredictable nature of user requests. To solve this problem, the study in [124] proposes a joint design of caching strategy and resource allocation in UAV networks. Specifically, the problem is formulated as an optimization problem aimed at minimizing the average delay of content downloading, which is proven to be NP-hard. To address this, the problem is first modeled as a MDP, and then dual-clip PPO is proposed. The algorithm adaptively determines the caching strategy, UAV trajectory, and transmission power, balancing exploration and exploitation by optimizing the strategies. Its problem objective is to minimize the average content download delay, while the goal of DRL is to maximize cumulative rewards. The solution obtained by the DRL algorithm is consistent with the objective.

Dynamic caching optimization for general content delivery is addressed in [124]. With the growing demand for video content, [125] extends the work by introducing a video-adaptive caching solution. By combining LSTM networks with PPO, this study optimizes UAV trajectories and caching decisions specifically for video content, responding to the highly dynamic nature of video requests. The proposed method significantly improves system performance, ensuring faster content delivery and reduced latency in video streaming. The objective in the problem formulation is to minimize the total system service delay over time slots, while the goal of DRL is to maximize cumulative reward. The solution obtained by the DRL algorithm is consistent with the objective solution of this paper.

B. Caching With UAV Interaction

For caching optimization with UAV interaction, [126] utilizes PPO algorithms, while [127] and [128] apply DDPG algorithms.

PPO based Caching: In [126], in the complex environment of a multi-UAV collaborative caching network, traditional content retrieval methods face challenges such as high latency and bandwidth bottlenecks. To address this challenge, this paper proposes a multi-UAV collaborative caching architecture in a NOMA manner, utilizing the mobility and cooperation capabilities of UAVs. The architecture, combined with DRL methods, optimizes caching decisions and flight trajectory planning to reduce content retrieval delay and improve content hit ratio. As illustrated in Fig. 7, HAP, UAVs 1-3, users, and the cloud are involved in the scenario. Based on the data cached by the UAVs, users 1 and 2 have three request paths: local UAV 2, neighboring UAVs 1 and 3, and the cloud, with the HAP acting as an intermediary for data transmission.

Its objective in the problem formulation is to maximize the long-term weighted content hit ratio. The solution obtained

by the PPO based proposal is consistent with the in the formulation. The reward is set as the total content retrieval delay for all MUs in each time slot. However, the paper does not specifically describe methods for enforcing the constraints. Here are several approaches to implementing these constraints. To implement the binary caching decision constraint, a discrete action space is defined where caching decisions take binary values (0 or 1). This is achieved using activation functions like sigmoid or softmax. To ensure the caching capacity constraint, penalty terms are applied for exceeding the UAV's capacity, or caching capacity is modeled as a state variable adjusted via optimization. For the content retrieval mode constraint, softmax functions in the action space ensure that only one retrieval mode (edge, cooperative, or cloud) is selected per user per time slot.

DDPG based Caching: The study in [126] addresses the optimization of caching and UAV trajectory planning in multi-UAV collaborative networks, but it mainly focuses on fixed spectrum resource allocation. Reference [127] introduces an air-to-air-to-ground edge computing network supporting clustered NOMA, which faces challenges such as spectrum resource scarcity and dynamic network environments. To tackle these issues, [127] proposes an architecture based on clustered non-orthogonal multiple access, utilizing satellites and multiple UAVs as collaborative edge servers. This approach, combined with DRL methods, optimizes data caching and computation offloading, thereby reducing system latency. As illustrated in the middle of Fig. 7, UAVs 4-6, users, and the cloud (satellite) are involved in the scenario. Users offload tasks to UAV 5, or to UAVs 4 and 6, or the cloud, based on cached strategies.

The objective in the problem formulation is to minimize the average system delay over the entire time period. The solution obtained by the DRL algorithm aligns with the objective. Considering the constraints on task offloading, program caching, and UAV distance, penalty terms are added to the reward function to minimize system delay. The task offloading and program caching constraints both involve binary decisions. The task offloading constraint ensures that each terminal can offload to only one UAV, while the program caching constraint ensures that the total cached program size at each UAV does not exceed its storage capacity. These constraints can be implemented using a discrete action space, where the network output is mapped to a range of zero to one using a softmax or sigmoid function.

The framework presented in [127] effectively addresses the challenges posed by dynamic network conditions and task offloading. However, it assumes static UAV positions, limiting its flexibility. Reference [128] explores a highly dynamic environment in which UAVs adjust their positions in real-time while also sharing their cached content, offering a more adaptable solution for reducing transmission delays in UAV-based networks.

The objective [128] is to minimize the total transmission delay for ground users in each time slot. The solution obtained by the DRL algorithm differs from the maximum reward per time slot. In addition, the paper does not specify the exact methods in the DRL framework for implementing these

constraints. For binary constraints, such as cache hit and user association decisions, use a discrete action space or binary selection, allowing the agent to choose the appropriate variables at each step. For linear constraints like cache capacity, transmission delay, maximum speed, and user limits, penalty terms are added to the reward function for violations. Position and coverage constraints are handled by a boundary filter, keeping UAVs within the service area. For collision avoidance, the DRL policy monitors UAV distances and adjusts trajectories to maintain a safe separation.

The objective in [128] is to minimize the total transmission delay for ground users in each time slot. However, the solution obtained by the DRL algorithm maximizes the cumulative reward across all time slots, which may differ from the objective of minimizing delay in a single time slot. Additionally, the paper does not specify the exact methods for implementing constraints within the DRL framework. For binary constraints, such as cache hit and user association decisions, a discrete action space or binary selection approach can be used. This allows the agent to choose the appropriate actions at each step, ensuring that decisions are made in a way that adheres to the binary nature of the problem. For linear constraints, such as cache capacity, transmission delay, maximum speed, and user limits, penalty terms are incorporated into the reward function to penalize constraint violations. These penalties guide the agent towards feasible solutions by discouraging actions that lead to violations. Position and coverage constraints are typically handled by using a boundary filter, which ensures that UAVs remain within the designated service area. For collision avoidance, it can be implemented by using a distance-based reward or penalty system.

VI. MULTI-AGENT DRL FOR UAV-ASSISTED NETWORKS

In UAV-assisted communication networks, MADRL has emerged as a powerful framework for addressing various optimization challenges, including UAV trajectory planning, energy minimization, task offloading, etc. The dynamic and complex nature of these networks has led to three distinct MADRL implementation approaches. In the basic approach, UAVs are modeled as homogeneous agents, focusing primarily on UAV-specific optimization objectives. However, given the complexity of communication systems involving multiple interacting entities (UAVs, users, and sometimes remote infrastructure like satellites), many studies adopt a heterogeneous agent modeling approach. This approach treats different system components as independent intelligent agents with distinct characteristics, objectives, and observational capabilities. For instance, UAVs may focus on trajectory and energy optimization, while users prioritize data transmission quality. Such heterogeneous modeling enables more realistic and efficient system-level optimization, though it requires advanced techniques like shared policies with individual adaptability and modular training strategies to balance different entities' constraints while ensuring coordination.

To further enhance training efficiency and convergence speed, the researchers developed a hybrid approach that decomposes the complex problem into two subproblems.

TABLE VI
MADRL COMPARISON

Work	Algorithm		Agent Type	Rewards	Action Constraints		
	DRL	Traditional			Direct Range	0-1 Combination	Others
[129]	CommNet-based MADRL	/	Homogeneous	Inconsistent	NS	NM	NM
[130]	MAPPO	/	Homogeneous	Inconsistent	NS	/	NM
[131]	MAPPO	/	Homogeneous	Inconsistent	NS	/	NM
[132]	TPCD-ICAGC (Actor-attention-critic)	/	Heterogeneous	Inconsistent	PR	/	NM
[133]	b-MAPPO	/	Heterogeneous	Inconsistent	NS	PR	PR, NS
[9]	MADQN	DM-ACO	Heterogeneous	Inconsistent	NS	PR, NS	/
[134]	MA-DuelingDQN	DIM	Homogeneous	Consistent	NS	/	NM
[135]	UA-MADDPG	GBCRA	Heterogeneous	Inconsistent	NS, PR	NS	NM

Homogeneous/Heterogeneous: Agents are categorized as homogeneous or heterogeneous, that is, whether they are modeled as identical or distinct entities.

One subproblem is typically addressed using traditional optimization methods, such as ant colony optimization (ACO) or dynamic programming (DP), which are particularly effective for specific constraints or static scenarios. The other subproblem employs a MADRL-based approach to handle dynamic environments and multi-agent interactions. While the results of the first subproblem often serve as input for the second subproblem, the order in which traditional optimization methods and MADRL algorithms are applied is not fixed. This hybrid approach combines the strengths of classical optimization and RL, achieving superior performance in highly dynamic and complex environments. Table VI provides a comprehensive overview of recent MADRL approaches, categorizing them according to agent homogeneity or heterogeneity, reward consistency, and action constraints, highlighting the flexibility of MADRL to address various challenges in UAV-assisted communication networks. As action constraints have been thoroughly examined in the preceding sections, this section focuses primarily on analyzing reward mechanisms and their implications for system optimization.

A. MADRL-Based Solutions

Reference [129] investigates dynamic and uncertain UAVs-assisted communication environments to optimize both energy efficiency and service reliability within three-dimensional network spaces. It proposes a MADRL framework based on CommNet architecture for optimizing UAV positioning and resource allocation. The state space design encompasses critical parameters including three-dimensional UAV positions, remaining energy levels, and service status indicators like UE connectivity and QoS metrics.

The reward function is structured with two primary components: individual rewards and collective rewards. Individual rewards incentivize energy-efficient UAV operations and high-quality service delivery through the consideration of remaining energy levels and user QoS metrics, while collective rewards enhance system-wide reliability by maximizing user coverage rates and minimizing inter-UAV coverage overlap. The framework implements penalties for inefficient energy utilization and redundant coverage overlap to ensure system stability and optimal performance. The action space comprises seven discrete movements, including variations in x , y , and z coordinates and hovering capabilities, with action constraints directly embedded within the network architecture.

Although the study does not explicitly address UAV collision avoidance mechanisms, it approaches this objective indirectly through a comprehensive state space design incorporating positional information of all UAVs and penalty components in the reward function for coverage overlap. However, this approach presents two significant limitations: the proposed design can not completely prevent UAV collisions, and the discrete action space implementation is inconsistent with the continuous action requirements of UAVs in real-world environments.

In contrast to [129], both [130] and [131] implement continuous action spaces in their approaches. Reference [130] introduces an emergency scenario force model (ESFM) for post-disaster emergency communication scenarios. In this model, GUs navigate towards their destinations along optimal paths while considering obstacle avoidance, injury treatment events, and so on. Under this dynamic ESFM scenario, the authors employ the MAPPO algorithm to determine UAV movements, aiming to maximize GU coverage. The study further evaluates the network performance trained with MAPPO using two distinct reward functions: one considering system fairness and another without fairness considerations. Reference [131] additionally incorporates optimization of UAV energy consumption. Both studies implement flight zone restrictions for UAVs within their network structures, with [130] specifically incorporating penalties in the reward function for UAVs that are too close to each other.

Unlike [129], [130], and [131], both [132] and [133] extend their consideration beyond UAV systems to encompass user systems, modeling UAVs and users as two heterogeneous types of agents. In [132], GUs primarily focus on network QoS, with their reward function designed as a function of throughput, while UAVs consider both throughput and system fairness. Furthermore, UAVs and GUs operate with distinct observation and action spaces. GUs only need to determine which UAV to connect to, whereas UAVs consider both their movement velocity and bandwidth allocation for GU service provision. Regarding constraints on UAV flight boundaries, inter-UAV distances, and UAV-GU distances, this study directly incorporates them as penalty terms within the reward function. A particularly noteworthy aspect is that due to the divergent objectives between UAVs and GUs, combined with the state complexity, the study introduces an attention architecture, forming an actor-attention-critic network architecture. This enhancement helps agents (either UAVs

or GUs) focus on information from other agents relevant to their decision-making, thereby optimizing collaborative capabilities.

In [133], the authors address a task offloading problem in UAV-assisted MEC networks, where GUs with limited computational capabilities frequently need to offload tasks to edge clouds for auxiliary computing. The study aims to optimize energy consumption across the entire system comprising both UAVs and GUs. For GU agents, energy consumption occurs primarily in the computation and communication aspects, which constitute the main components of the GU reward function. Additionally, the study incorporates QoS considerations by introducing a penalty term in the reward function for tasks that fail to meet latency requirements. For UAVs, energy consumption predominantly involves computation and mobility. The UAV reward function design includes not only energy consumption but also incorporates three penalty terms: failure to meet GU latency requirements, insufficient inter-UAV spacing, and UAV flight beyond service area boundaries. Regarding action spaces, GUs must decide both which UAV to allocate tasks to and the quantity of tasks to allocate, while UAVs must determine their movement trajectories and CPU frequency allocation for processing different tasks. To address this complex problem involving both binary and continuous variables, the authors employ the b-MAPPO algorithm. They enhance the traditional MAPPO's Gaussian distribution output by implementing Beta distribution to improve boundary condition adaptability and policy flexibility.

B. Hybrid MADRL Solutions: Combining Traditional Optimization

In contrast to Section VI-A, [9] not only employs heterogeneous agent modeling but also decomposes the problem into two sub-problems during the modeling process. They first solve one sub-problem using traditional optimization methods, then incorporate its results along with the second sub-problem into a MADRL-based algorithm. Reference [9] investigates content caching optimization in autonomous vehicular networks supported by a non-terrestrial network comprising low earth orbit (LEO) satellites and UAVs. In dynamic and complex communication environments, the study addresses the optimization of caching strategies under heterogeneous cache hardware and frequent network dynamics to reduce system propagation delay and improve cache hit ratios.

The authors propose a MADRL-based hierarchical caching and asynchronous update scheme. This approach combines a discrete multi-objective ant colony optimization (DM-ACO) algorithm for selecting appropriate caching satellites with MADRL for hierarchical caching decisions and asynchronous updates. In implementing the MADQN algorithm, UAVs and LEO satellites are modeled as two distinct types of heterogeneous agents, with their action spaces encompassing decisions between content caching and replacement. To simplify the process, the study implements an asynchronous update mechanism, splitting content switching into two steps: discarding

existing content and caching new content. Regarding reward function design, the UAV-layer reward function encourages UAVs to prioritize caching content with high latency requirements and optimizes caching decisions based on the squared ratio of content transmission delay to threshold. The LEO layer, conversely, weights content access efficiency through cache hit ratio. It is worth noting that these reward functions are inconsistent with its objective function in the mathematical model.

Unlike [9], [134] addresses the joint optimization of task offloading and UAV trajectory in UAVs-assisted MEC networks, with particular emphasis on dynamic demand and service fairness optimization. The study proposes a multi-agent energy-efficient joint trajectory and computation offloading scheme, which achieves joint optimization of trajectory control and task offloading decisions through an optimization-embedding MADRL (OMADRL) algorithm. It employs MA-Dueling DQN to learn UAV trajectory policies, while offloading decisions are determined through a mixed integer non-linear programming approach (called Dinkelbach iteration method). These offloading decisions subsequently guide trajectory control, thereby reducing action space dimensionality and enhancing convergence efficiency. The study models all UAVs as homogeneous agents, with a discrete action space comprising 8 directional options and 4 velocity levels to describe flight trajectories in three-dimensional space. Additionally, the authors develop a "collision avoidance action adjustment mechanism" to prevent UAVs from approaching too closely to each other. Regarding task offloading, user tasks are categorized into delay-sensitive and delay-tolerant tasks, with sensitive tasks executed by the nearest UAV while tolerant tasks can be relayed to BS. The reward function, aligned with the objective function, incorporates fairness weights, delay tolerance, and energy efficiency, incentivizing UAVs to cover hotspot areas and optimize task allocation.

Reference [135] focuses on a MEC network with air-ground collaboration, specifically addressing task offloading for user vehicles (UVs) and resource allocation for MEC servers in ultra-reliable low latency communications (URLLC). The paper introduces an air-ground vehicular collaborative computing network model and employs Lyapunov optimization to decompose the original problem into two subproblems: transmission cost optimization and computational resource allocation optimization. Consequently, the authors propose a URLLC-aware MADDPG (UA-MADDPG) algorithm and a greedy resource allocation method. By utilizing UA-MADDPG to jointly optimize UAV trajectory, task offloading, and spectrum allocation, the approach effectively addresses uncertainties in global state information. The study considers all UAVs, UVs, and OPs as homogeneous agents, with agent actions encompassing task offloading decisions for UVs, UAV flight distance and direction decisions, and spectrum resource allocation decisions for OPs. The action space incorporates both continuous and discrete variables. The reward function simultaneously considers system transmission costs and URLLC queue stability constraints, with additional penalties imposed for scenarios.

VII. CHALLENGES AND OPEN ISSUES

While DRL has demonstrated significant potential in optimizing UAV communication networks, there remain several challenges that need to be addressed for its widespread implementation and further advancement. These challenges arise from the complexity of real-world environments, the computational demands of DRL algorithms, and the need for adaptable and scalable solutions.

A. High Dimensionality and Computational Complexity

UAV networks typically involve large-scale [136], dynamic environments with numerous interconnected nodes [137] and fluctuating network conditions. This complexity results in high-dimensional state and action spaces, which can challenge the efficiency of DRL algorithms. Training DRL models in such environments often requires significant computational resources and extended training times, making real-time applications difficult. Addressing this requires developing more computationally efficient algorithms and employing techniques such as function approximation, transfer learning, or model compression to reduce the overhead.

B. Scalability and Multi-Agent Coordination

As UAV networks grow in size, the need for scalable DRL algorithms becomes more pressing. MADRL approaches are required for coordination among multiple UAVs to optimize tasks like power allocation [137], channel assignment, and task offloading. However, as the number of agents increases, so does the complexity of learning effective policies due to the growing action spaces and interactions between agents. Solutions such as decentralized learning or hierarchical DRL architectures are necessary to ensure that these systems remain scalable without overwhelming computational resources [138].

C. Modeling Realistic Environments and Uncertainty

UAV communication networks operate in highly dynamic environments where factors such as mobility [139], signal interference, and changing weather conditions can significantly impact performance. Traditional DRL algorithms often assume stationary environments, which may not accurately reflect the real-world uncertainties faced by UAVs. Developing DRL methods that can robustly adapt to dynamic, non-stationary conditions remains a key challenge [140]. Approaches such as meta-RL and robust DRL have the potential to improve adaptability but require further research and validation.

D. Reward Function Design and Convergence

The success of DRL relies heavily on the design of appropriate reward functions that reflect the objectives of the UAV communication tasks. Poorly designed rewards can lead to suboptimal policies or even divergence during training. In complex UAV networks, where multiple objectives (such as minimizing latency, maximizing throughput, and ensuring energy efficiency) are involved, balancing conflicting rewards poses an additional challenge [138], [139]. Research is needed to explore better multi-objective optimization techniques and

methods for reward shaping to guide DRL agents toward optimal solutions efficiently.

E. Data Efficiency and Public Datasets

DRL algorithms often require a large volume of data to learn effective policies, making data efficiency a key challenge. Collecting real-world UAV network data is costly and time-consuming, and simulated environments may not always accurately reflect real-world conditions. Improving sample efficiency through techniques like model-based DRL or transfer learning can help address this issue by reducing the data needed for training [140]. Furthermore, the lack of publicly available datasets specific to UAV communication scenarios limits the ability to benchmark and compare DRL solutions. Creating shared datasets that capture UAV-specific tasks like power allocation and caching optimization is essential to accelerate research in this field.

VIII. CONCLUSION

This survey has examined how DRL can be applied to solve critical mathematical optimization models in UAV communications and networking. Traditional approaches struggle with the dynamic and complex nature of UAV networks, while DRL offers a more adaptable solution by learning to optimize over time. By focusing on system modeling and progressing to DRL implementation, we have outlined how key optimization problems, such as power allocation, channel assignment, and task offloading, can be framed as mathematical models and efficiently solved using DRL techniques. This progression from modeling to implementation serves as a practical guide for researchers and practitioners. Additionally, we have identified future challenges and opportunities for further advancing the application of DRL in UAV networks, particularly in terms of scalability and real-time adaptability. The potential in DRL to continuously learn and adapt to evolving network conditions makes it a crucial tool for future UAV communication systems.

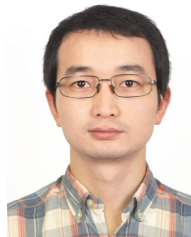
REFERENCES

- [1] J. Kang et al., "UAV-assisted dynamic avatar task migration for vehicular metaverse services: A multi-agent deep reinforcement learning approach," *IEEE/CAA J. Automatica Sinica*, vol. 11, no. 2, pp. 430–445, Feb. 2024.
- [2] N. T. Hoa, D. Van Dai, L. H. Lan, N. C. Luong, D. Van Le, and D. Niyato, "Deep reinforcement learning for multi-hop offloading in UAV-assisted edge computing," *IEEE Trans. Veh. Technol.*, vol. 72, no. 12, pp. 16917–16922, Dec. 2023.
- [3] J. Kang et al., "Tiny multiagent DRL for twins migration in UAV metaverses: A multi-leader multi-follower Stackelberg game approach," *IEEE Internet Things J.*, vol. 11, no. 12, pp. 21021–21036, Jun. 2024.
- [4] K. K. Nguyen, T. Q. Duong, T. Do-Duy, H. Claussen, and L. Hanzo, "3-D UAV trajectory and data collection optimisation via deep reinforcement learning," *IEEE Trans. Commun.*, vol. 70, no. 4, pp. 2358–2371, Apr. 2022.
- [5] X. Yuan, F. Tang, M. Zhao, and N. Kato, "Joint rate and coverage optimization for the THZ/RF multi-band communications of space-air-ground integrated network in 6G," *IEEE Trans. Wireless Commun.*, vol. 23, no. 6, pp. 6669–6682, Jun. 2024.
- [6] J. Du et al., "MaDDPG-based joint service placement and task offloading in MEC empowered air-ground integrated networks," *IEEE Internet Things J.*, vol. 11, no. 6, pp. 10600–10615, Mar. 2024.
- [7] Z. Liu, C. Zhan, Y. Cui, C. Wu, and H. Hu, "Robust edge computing in UAV systems via scalable computing and cooperative computing," *IEEE Wireless Commun.*, vol. 28, no. 5, pp. 36–42, Oct. 2021.

- [8] Y. Qu et al., "Elastic collaborative edge intelligence for UAV swarm: Architecture, challenges, and opportunities," *IEEE Commun. Mag.*, vol. 62, no. 1, pp. 62–68, Jan. 2024.
- [9] B. Mao et al., "On a hierarchical content caching and asynchronous updating scheme for non-terrestrial network-assisted connected automated vehicles," *IEEE J. Sel. Areas Commun.*, vol. 43, no. 1, pp. 64–74, Jan. 2025.
- [10] F. Tang, X. Chen, M. Zhao, and N. Kato, "The roadmap of communication and networking in 6G for the metaverse," *IEEE Wireless Commun.*, vol. 30, no. 4, pp. 72–81, Aug. 2023.
- [11] J. Xu et al., "IRs-UAV assisted secure integrated sensing and communication," *IEEE Wireless Commun.*, vol. 31, no. 5, pp. 61–67, Aug. 2024.
- [12] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: MIT Press, 2018.
- [13] X. Xu, L. Zuo, and Z. Huang, "Reinforcement learning algorithms with function approximation: Recent advances and applications," *Inf. Sci.*, vol. 261, pp. 1–31, Jul. 2014.
- [14] H. Kurunathan, H. Huang, K. Li, W. Ni, and E. Hossain, "Machine learning-aided operations and communications of unmanned aerial vehicles: A contemporary survey," *IEEE Commun. Surveys Tuts.*, vol. 26, no. 1, pp. 496–533, 1st Quart., 2023.
- [15] Q. Chen, Z. Guo, W. Meng, S. Han, C. Li, and T. Q. Quek, "A survey on resource management in joint communication and computing-embedded SAGIN," *IEEE Commun. Surveys Tuts.*, vol. 27, no. 3, pp. 1911–1954, Jun. 2025.
- [16] F. Frattolillo, D. Brunori, and L. Iocchi, "Scalable and cooperative deep reinforcement learning approaches for multi-UAV systems: A systematic review," *Drones*, vol. 7, no. 4, p. 236, 2023.
- [17] P. Cao et al., "Computational intelligence algorithms for UAV swarm networking and collaboration: A comprehensive survey and future directions," *IEEE Commun. Surveys Tuts.*, vol. 26, no. 4, pp. 2684–2728, 4th Quart., 2024.
- [18] K. Teixeira, G. Miguel, H. S. Silva, and F. Madeiro, "A survey on applications of unmanned aerial vehicles using machine learning," *IEEE Access*, vol. 11, pp. 117582–117621, 2023.
- [19] Z. Ning et al., "Mobile edge computing and machine learning in the Internet of Unmanned Aerial Vehicles: A survey," *ACM Comput. Surveys*, vol. 56, no. 1, pp. 1–31, 2023.
- [20] T. Naous, M. Itani, M. Awad, and S. Sharafeddine, "Reinforcement learning in the sky: A survey on enabling intelligence in NTN-based communications," *IEEE Access*, vol. 11, pp. 19941–19968, 2023.
- [21] P. Zhang et al., "UAV communication in space-air-ground integrated networks (SAGINs): Technologies, applications, and challenges," *Drones*, vol. 9, no. 2, p. 22, 2025.
- [22] Y. Ding, Z. Yang, Q.-V. Pham, Y. Hu, Z. Zhang, and M. Shikh-Bahaei, "Distributed machine learning for UAV swarms: Computing, sensing, and semantics," *IEEE Internet Things J.*, vol. 11, no. 5, pp. 7447–7473, Mar. 2024.
- [23] Y. Bai, H. Zhao, X. Zhang, Z. Chang, R. Jäntti, and K. Yang, "Toward autonomous multi-UAV wireless network: A survey of reinforcement learning-based approaches," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 4, pp. 3038–3067, 4th Quart., 2023.
- [24] Z. Kaleem, F. A. Orakzai, W. Ishaq, K. Latif, J. Zhao, and A. Jamalipour, "Emerging trends in UAVs: From placement, semantic communications to generative AI for mission-critical networks," *IEEE Trans. Consum. Electron.*, early access, Jul. 29, 2024, doi: 10.1109/TCE.2024.3434971.
- [25] T. Li et al., "Applications of multi-agent reinforcement learning in future Internet: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 24, no. 2, pp. 1240–1279, 2nd Quart., 2022.
- [26] J. Tan, X. Dai, F. Tang, M. Zhao, and N. Kato, "Intelligent configuration method based on UAV-driven frequency selective surface for communication band shielding," *IEEE Internet Things J.*, vol. 10, no. 13, pp. 11507–11517, Jul. 2023.
- [27] W. Y. B. Lim et al., "Towards federated learning in UAV-enabled Internet of Vehicles: A multi-dimensional contract-matching approach," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 8, pp. 5140–5154, Aug. 2021.
- [28] Q. Wang, Z. Chen, J. Cai, and Z. Tian, "Establishing UAV-aided VBSS for secure multicasting," *IEEE Wireless Commun. Lett.*, vol. 9, no. 7, pp. 1009–1013, Jul. 2020.
- [29] G. Sun et al., "Joint task offloading and resource allocation in aerial-terrestrial UAV networks with edge and fog computing for post-disaster rescue," *IEEE Trans. Mobile Comput.*, vol. 23, no. 9, pp. 8582–8600, Sep. 2024.
- [30] Q. Wu, Y. Zeng, and R. Zhang, "Joint trajectory and communication design for multi-UAV enabled wireless networks," *IEEE Trans. Wireless Commun.*, vol. 17, no. 3, pp. 2109–2121, Mar. 2018.
- [31] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [32] H. Hasselt, "Double Q -learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 23, 2010, pp. 1–9.
- [33] Z. Wang, T. Schaul, M. Hessel, H. Hasselt, M. Lanctot, and N. Freitas, "Dueling network architectures for deep reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 1995–2003.
- [34] M. Hausknecht and P. Stone, "Deep recurrent Q -learning for partially observable MDPS," in *Proc. AAAI Fall Symp. Series*, 2015, pp. 29–37.
- [35] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," in *Proc. ICLR*, 2016, pp. 1–6.
- [36] M. G. Bellemare, W. Dabney, and R. Munos, "A distributional perspective on reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 449–458.
- [37] W. Dabney, M. Rowland, M. Bellemare, and R. Munos, "Distributional reinforcement learning with quantile regression," in *Proc. AAAI Conf. Artif. Intell.*, vol. 32, 2018, pp. 2892–2901.
- [38] W. Dabney, G. Ostrovski, D. Silver, and R. Munos, "Implicit quantile networks for distributional reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1104–1113.
- [39] M. Fortunato et al., "Noisy networks for exploration," in *Proc. ICLR (Poster)*, 2018, pp. 1–9.
- [40] Z. Hu, K. Wan, X. Gao, Y. Zhai, and Q. Wang, "Deep reinforcement learning approach with multiple experience pools for UAV's autonomous motion planning in complex unknown environments," *Sensors*, vol. 20, no. 7, p. 1890, 2020.
- [41] J. F. Hernandez-Garcia and R. S. Sutton, "Understanding multi-step deep reinforcement learning: A systematic study of the DQN target," 2019, *arXiv:1901.07510*.
- [42] M. Hessel et al., "Rainbow: Combining improvements in deep reinforcement learning," in *Proc. AAAI Conf. Artif. Intell.*, 2018, pp. 1–7.
- [43] A. Nair et al., "Massively parallel methods for deep reinforcement learning," 2015. [Online]. Available: <https://arxiv.org/abs/1507.04296>
- [44] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour, "Policy gradient methods for reinforcement learning with function approximation," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 12, 1999, pp. 1057–1063.
- [45] S. M. Kakade, "A natural policy gradient," in *Advances in Neural Information Processing Systems*, vol. 14. Cambridge, MA, USA: MIT Press, 2001.
- [46] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, "Trust region policy optimization," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 1–7.
- [47] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017. [Online]. Available: <https://arxiv.org/abs/1707.06347>
- [48] Z. Wang et al., "Sample efficient actor-critic with experience replay," 2016. [Online]. Available: <https://arxiv.org/abs/1611.01224>
- [49] Y. Wu, E. Mansimov, R. B. Grosse, S. Liao, and J. Ba, "Scalable trust-region method for deep reinforcement learning using Kronecker-factored approximation," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 5279–5288.
- [50] V. Konda and J. Tsitsiklis, "Actor-critic algorithms," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 12, 1999, pp. 1–9.
- [51] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller, "Deterministic policy gradient algorithms," in *Proc. Int. Conf. Mach. Learn.*, 2014, pp. 387–395.
- [52] T. P. Lillicrap et al., "Continuous control with deep reinforcement learning," in *Proc. ICLR*, 2016, pp. 1–8.
- [53] S. Fujimoto, H. Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1–6.
- [54] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1856–1865.
- [55] L. Espeholt et al., "Impala: Scalable distributed deep-RL with importance weighted actor-learner architectures," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1–9.
- [56] M. Babaeizadeh, I. Frosio, S. Tyree, J. Clemons, and J. Kautz, "Reinforcement learning through asynchronous advantage actor-critic on a GPU," in *Proc. ICLR (Poster)*, 2017, pp. 1–7.
- [57] C. Alfredo, C. Humberto, and C. Arjun, "Efficient parallel methods for deep reinforcement learning," in *Proc. Multidiscipl. Conf. Reinforcement Learn. Decision Making (RLDM)*, 2017, pp. 1–9.

- [58] M. Jaderberg et al., "Reinforcement learning with unsupervised auxiliary tasks," in *Proc. ICLR*, 2017, p. 6.
- [59] B. O'Donoghue, R. Munos, K. Kavukcuoglu, and V. Mnih, "Combining policy gradient and Q -learning," in *Proc. ICLR (Poster)*, 2017, p. 9.
- [60] A. Gruslys, W. Dabney, M. G. Azar, B. Piot, M. Bellemare, and R. Munos, "The reactor: A fast and sample-efficient actor-critic agent for reinforcement learning," in *Proc. Int. Conf. Learn. Rep.*, 2018, pp. 1–7.
- [61] R. Munos, T. Stepleton, A. Harutyunyan, and M. Bellemare, "Safe and efficient off-policy reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 29, 2016, pp. 1046–1054.
- [62] J. N. Foerster, R. Y. Chen, M. Al-Shedivat, S. Whiteson, P. Abbeel, and I. Mordatch, "Learning with opponent-learning awareness." 2017. [Online]. Available: <https://arxiv.org/abs/1709.04326>
- [63] C. S. de Witt et al., "Is independent learning all you need in the starcraft multi-agent challenge?" 2020. [Online]. Available: <http://arxiv.org/abs/2011.09533>
- [64] J. Foerster, G. Farquhar, T. Afouras, N. Nardelli, and S. Whiteson, "Counterfactual multi-agent policy gradients," in *Proc. AAAI Conf. Artif. Intell.*, vol. 32, 2018, pp. 2974–2982.
- [65] G. Tesauro, "Extending Q -learning to general adaptive multi-agent systems," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 16, 2003, pp. 871–878.
- [66] C. Claus and C. Boutilier, "The dynamics of reinforcement learning in cooperative multiagent systems," in *Proc. AAAI/IAAI*, 1998, pp. 746–752.
- [67] M. Tan, "Multi-agent reinforcement learning: Independent vs. cooperative agents," in *Proc. 10th Int. Conf. Mach. Learn.*, 1993, pp. 1–9.
- [68] P. Sunehag et al., "Value-decomposition networks for cooperative multi-agent learning based on team reward," in *Proc. 17th Int. Conf. Auton. Agents Multiagent Syst.*, 2018, pp. 2085–2087.
- [69] T. Rashid, M. Samvelyan, C. S. De Witt, G. Farquhar, J. Foerster, and S. Whiteson, "Monotonic value function factorisation for deep multi-agent reinforcement learning," *J. Mach. Learn. Res.*, vol. 21, no. 178, pp. 1–51, 2020.
- [70] K. Son, D. Kim, W. J. Kang, D. E. Hostallero, and Y. Yi, "QTRan: Learning to factorize with transformation for cooperative multi-agent reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 5887–5896.
- [71] T. Wang, T. Gupta, A. Mahajan, B. Peng, S. Whiteson, and C. Zhang, "RODE: Learning roles to decompose multi-agent tasks," in *Proc. Int. Conf. Learn. Rep.*, 2021, pp. 1–9.
- [72] J. Wang, Z. Ren, T. Liu, Y. Yu, and C. Zhang, "QPLEX: Duplex dueling multi-agent Q -learning," in *Proc. Int. Conf. Learn. Rep.*, 2021, pp. 1–8.
- [73] R. Lowe, Y. I. Wu, A. Tamar, J. Harb, O. P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 6379–6390.
- [74] C. Yu et al., "The surprising effectiveness of ppo in cooperative multi-agent games," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 24611–24624.
- [75] M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Hoboken, NJ, USA: Wiley, 2014.
- [76] C. J. Watkins and P. Dayan, " Q -learning," *Mach. Learn.*, vol. 8, pp. 279–292, Jun. 1992.
- [77] A. T. Azar et al., "Drone deep reinforcement learning: A review," *Electronics*, vol. 10, no. 9, p. 999, 2021.
- [78] D. Zha et al., "Douzero: Mastering Doudizhu with self-play deep reinforcement learning," in *Proc. 38th Int. Conf. Mach. Learn.*, 2021, pp. 12333–12344.
- [79] H. P. Van Hasselt, A. Guez, M. Hessel, V. Mnih, and D. Silver, "Learning values across many orders of magnitude," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 29, 2016, pp. 4287–4295.
- [80] O. Anschel, N. Baram, and N. Shimkin, "Averaged-DQN: Variance reduction and stabilization for deep reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 1–8.
- [81] T. Hester et al., "Deep Q -learning from demonstrations," in *Proc. AAAI Conf. Artif. Intell.*, vol. 32, 2018, pp. 3223–3230.
- [82] J. Schulman, X. Chen, and P. Abbeel, "Equivalence between policy gradients and soft Q -learning," 2017, [arXiv:1704.06440](https://arxiv.org/abs/1704.06440).
- [83] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine Learning*, vol. 8, pp. 229–256, 1992.
- [84] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in *Proc. Int. Conf. Learn. Rep. (ICLR)*, 2016, pp. 1–8.
- [85] V. Mnih et al., "Asynchronous methods for deep reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 1–8.
- [86] W. Zhao, Y. Cheng, Z. Liu, X. Wu, and N. Kato, "Asynchronous DRL based multi-hop task offloading in RSU-assisted IoV networks," *IEEE Trans. Cogn. Commun. Netw.*, vol. 11, no. 1, pp. 546–555, Feb. 2025.
- [87] W. Zhao et al., "DRL connects Lyapunov in delay and stability optimization for offloading proactive sensing tasks of RSUS," *IEEE Trans. Mobile Comput.*, vol. 23, no. 7, pp. 7969–7982, Jul. 2024.
- [88] Z. He, W. Qiu, W. Zhao, X. Shao, and Z. Liu, "Understanding world models through multi-step pruning policy via reinforcement learning," *Inf. Sci.*, vol. 686, Jan. 2025, Art. no. 121361.
- [89] M. L. Littman, "Markov games as a framework for multi-agent reinforcement learning," in *Proc. Mach. Learn.*, 1994, pp. 157–163.
- [90] P. Hernandez-Leal, B. Kartal, and M. E. Taylor, "Is multiagent deep reinforcement learning the answer or the question? A brief survey," *Learning*, vol. 21, p. 22, Jun. 2018.
- [91] M. Lauer and M. A. Riedmiller, "An algorithm for distributed reinforcement learning in cooperative multi-agent systems," in *Proc. 17th Int. Conf. Mach. Learn.*, 2000, pp. 535–542.
- [92] L. Matignon, G. J. Laurent, and N. Le Fort-Piat, "Independent reinforcement learners in cooperative Markov games: A survey regarding coordination problems," *Knowl. Eng. Rev.*, vol. 27, no. 1, pp. 1–31, 2012.
- [93] M. Bowling, "Convergence problems of general-sum multiagent reinforcement learning," in *Proc. ICML*, 2000, pp. 89–94.
- [94] J. Hu and M. P. Wellman, "Multiagent reinforcement learning: Theoretical framework and an algorithm," in *Proc. ICML*, 1998, pp. 1–6.
- [95] M. L. Littman, "Friend-or-foe Q -learning in general-sum games," in *Proc. ICML*, 2001, pp. 1–6.
- [96] K. Ciosek and S. Whiteson, "Offer: Off-environment reinforcement learning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 31, 2017, pp. 1819–1825.
- [97] J. Foerster et al., "Stabilising experience replay for deep multi-agent reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 1146–1155.
- [98] J. K. Gupta, M. Egorov, and M. Kochenderfer, "Cooperative multi-agent control using deep reinforcement learning," in *Proc. Autonomous Agents Multiagent Syst. (AAMAS Workshops)*, 2017, pp. 66–83.
- [99] H. Hu and J. N. Foerster, "Simplified action decoder for deep multi-agent reinforcement learning," in *Proc. ICLR*, 2020, pp. 1–8.
- [100] M. Andrychowicz et al., "What matters for on-policy deep actor-critic methods? A large-scale study," in *Proc. Int. Conf. Learn. Rep.*, 2020, pp. 1–8.
- [101] L. Engstrom et al., "Implementation matters in deep RL: A case study on PPO and TRPO," in *Proc. Int. Conf. Learn. Rep.*, 2019, pp. 1–9.
- [102] A. Ilyas et al., "A closer look at deep policy gradients," in *Proc. Int. Conf. Learn. Rep.*, 2020, pp. 1–9.
- [103] G. Tucker, S. Bhupatiraju, S. Gu, R. Turner, Z. Ghahramani, and S. Levine, "The mirage of action-dependent baselines in reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1–8.
- [104] J. Li, C. Zhou, J. Liu, M. Sheng, N. Zhao, and Y. Su, "Reinforcement learning-based resource allocation for coverage continuity in high dynamic UAV communication networks," *IEEE Trans. Wireless Commun.*, vol. 23, no. 2, pp. 848–860, Jun. 2024.
- [105] Y. Liu, J. Yan, and X. Zhao, "Deep-reinforcement-learning-based optimal transmission policies for opportunistic uav-aided wireless sensor network," *IEEE Internet Things J.*, vol. 9, no. 15, pp. 13823–13836, Aug. 2022.
- [106] R. Zhong, X. Liu, Y. Liu, and Y. Chen, "Multi-agent reinforcement learning in NOMA-aided UAV networks for cellular offloading," *IEEE Trans. Wireless Commun.*, vol. 21, no. 3, pp. 1498–1512, Mar. 2022.
- [107] W. Qi, Q. Song, L. Guo, and A. Jamalipour, "Energy-efficient resource allocation for UAV-assisted vehicular networks with spectrum sharing," *IEEE Trans. Veh. Technol.*, vol. 71, no. 7, pp. 7691–7702, Jul. 2022.
- [108] C. Wang, D. Deng, L. Xu, and W. Wang, "Resource scheduling based on deep reinforcement learning in UAV assisted emergency communication networks," *IEEE Trans. Commun.*, vol. 70, no. 6, pp. 3834–3848, Jun. 2022.
- [109] R. Tang, J. Wang, Y. Zhang, F. Jiang, X. Zhang, and J. Du, "Throughput maximization in NOMA enhanced RIS-assisted multi-UAV networks: A deep reinforcement learning approach," *IEEE Trans. Veh. Technol.*, vol. 74, no. 1, pp. 730–745, Jan. 2025.
- [110] Q. Zhang, J. Miao, Z. Zhang, F. R. Yu, F. Fu, and T. Wu, "Energy-efficient video streaming in UAV-enabled wireless networks: A safe-DQN approach," in *Proc. GLOBECOM*, 2020, pp. 1–7.

- [111] X. Li, S. Cheng, H. Ding, M. Pan, and N. Zhao, "When UAVs meet cognitive radio: Offloading traffic under uncertain spectrum environment via deep reinforcement learning," *IEEE Trans. Wireless Commun.*, vol. 22, no. 2, pp. 824–837, Feb. 2023.
- [112] K. K. Nguyen, S. R. Khosravirad, D. B. da Costa, L. D. Nguyen, and T. Q. Duong, "Reconfigurable intelligent surface-assisted multi-UAV networks: Efficient resource allocation with deep reinforcement learning," *IEEE J. Sel. Topics Signal Process.*, vol. 16, no. 3, pp. 358–368, Apr. 2022.
- [113] R. Ding, F. Gao, and X. S. Shen, "3D UAV trajectory design and frequency band allocation for energy-efficient and fair communication: A deep reinforcement learning approach," *IEEE Trans. Wireless Commun.*, vol. 19, no. 12, pp. 7796–7809, Dec. 2020.
- [114] S. Yin and F. R. Yu, "Resource allocation and trajectory design in UAV-aided cellular networks based on multiagent reinforcement learning," *IEEE Internet Things J.*, vol. 9, no. 4, pp. 2933–2943, Feb. 2022.
- [115] B. Yin, X. Fang, and X. Wang, "Joint optimization of trajectory control, resource allocation, and user association based on DRL for multi-fixed-wing UAV networks," *IEEE Trans. Wireless Commun.*, vol. 23, no. 10, pp. 13330–13342, Oct. 2024.
- [116] D. S. Lakew, A. T. Tran, N. N. Dao, and S. Cho, "Intelligent offloading and resource allocation in heterogeneous aerial access IoT networks," *IEEE Internet Things J.*, vol. 10, no. 7, pp. 5704–5718, Apr. 2023.
- [117] A. Termehchi, A. Syed, W. S. Kennedy, and M. Erol-Kantarci, "Distributed safe multi-agent reinforcement learning: Joint design of THz-enabled UAV trajectory and channel allocation," *IEEE Trans. Veh. Technol.*, vol. 73, no. 10, pp. 12856–12870, Oct. 2024.
- [118] K. Lin, H. Yang, M. Zheng, L. Xiao, C. Huang, and D. Niyato, "Penalized reinforcement learning-based energy-efficient UAV-RIS assisted maritime uplink communications against jamming," *IEEE Trans. Veh. Technol.*, vol. 73, no. 10, pp. 15768–15773, Oct. 2024.
- [119] H. M. Hariz, S. S. Z. Mosaddegh, N. Mokari, M. R. Javan, B. A. Arand, and E. A. Jorswieck, "AI-based radio resource management and trajectory design for IRS-UAV-assisted PD-NOMA communication," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 3, pp. 3385–3400, Jun. 2024.
- [120] K. K. Nguyen, A. Masaracchia, V. Sharma, H. V. Poor, and T. Q. Duong, "RIS-assisted UAV communications for IoT with wireless power transfer using deep reinforcement learning," *IEEE J. Sel. Topics Signal Process.*, vol. 16, no. 5, pp. 1086–1096, Aug. 2022.
- [121] J. Huang, M. Zhang, J. Wan, Y. Chen, and N. Zhang, "Joint data caching and computation offloading in UAV-assisted Internet of Vehicles via federated deep reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 73, no. 11, pp. 17644–17656, Nov. 2024.
- [122] N. Lin, X. Han, A. Hawbani, Y. Sun, Y. Guan, and L. Zhao, "Deep reinforcement learning based dual-timescale service caching and computation offloading for multi-UAV assisted MEC systems," *IEEE Trans. Netw. Service Manag.*, vol. 22, no. 1, pp. 605–617, Feb. 2025.
- [123] J. Ji, K. Zhu, and L. Cai, "Trajectory and communication design for cache-enabled UAVs in cellular networks: A deep reinforcement learning approach," *IEEE Trans. Mobile Comput.*, vol. 22, no. 10, pp. 6190–6204, Oct. 2023.
- [124] J. Chen, F. Tan, H. Chen, and S. Li, "Caching strategy and resource allocation in cache-enabled UAV networks: A deep reinforcement learning approach," in *Proc. IEEE/CIC Int. Conf. Commun. China*, 2024, pp. 1–8.
- [125] X. Ren, X. Chen, L. Jiao, X. Dai, and Z. Dong, "Joint optimization of UAV trajectory planning, video cache placement and transcoding in UAV-assisted 6G networks: A PPO-L based approach," in *Proc. 27th Int. Conf. Comput. Supported Cooperat. Work Design*, 2024, pp. 1310–1315.
- [126] P. Qin, Y. Fu, J. Zhang, S. Geng, J. Liu, and X. Zhao, "DRL-based resource allocation and trajectory planning for noma-enabled multi-UAV collaborative caching 6G network," *IEEE Trans. Veh. Technol.*, vol. 73, no. 6, pp. 8750–8764, Jun. 2024.
- [127] P. Qin, M. Fu, Y. Fu, R. Ding, and X. Zhao, "Collaborative edge computing and program caching with routing plan in C-NOMA-enabled space-air-ground network," *IEEE Trans. Wireless Commun.*, vol. 23, no. 12, pp. 18302–18315, Dec. 2024.
- [128] H. Muslih, S. A. Kazmi, M. Mazzara, and G. Baye, "Cache sharing in UAV-enabled cellular network: A deep reinforcement learning-based approach," *IEEE Access*, vol. 12, pp. 43422–43435, 2024.
- [129] J. Kim, S. Park, S. Jung, and C. Cordeiro, "Cooperative multi-UAV positioning for aerial Internet service management: A multi-agent deep reinforcement learning approach," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 4, pp. 3797–3812, Aug. 2024.
- [130] W. Qiu, X. Shao, S. W. Loke, Z. He, F. Alqahtani, and H. Masui, "Enhancing UAV communications in disasters: Integrating ESFM and MAPPO for superior performance," *J. Circuits Syst. Comput.*, vol. 34, no. 5, 2025, Art. no. 2550112.
- [131] W. Qiu, X. Shao, H. Masui, and W. Liu, "Optimizing drone energy use for emergency communications in disasters via deep reinforcement learning," *Future Internet*, vol. 16, no. 7, p. 245, 2024.
- [132] Z. Lu, Z. Jia, Q. Wu, and Z. Han, "Joint trajectory planning and communication design for multiple UAVs in intelligent collaborative air-ground communication systems," *IEEE Internet Things J.*, vol. 11, no. 19, pp. 31053–31067, Oct. 2024.
- [133] B. Li, R. Yang, L. Liu, J. Wang, N. Zhang, and M. Dong, "Robust computation offloading and trajectory optimization for multi-UAV-assisted MEC: A multiagent DRL approach," *IEEE Internet Things J.*, vol. 11, no. 3, pp. 4775–4786, Feb. 2024.
- [134] X. Li, X. Du, N. Zhao, and X. Wang, "Computing over the sky: Joint UAV trajectory and task offloading scheme based on optimization-embedding multi-agent deep reinforcement learning," *IEEE Trans. Commun.*, vol. 72, no. 3, pp. 1355–1369, Mar. 2024.
- [135] P. Qin, Y. Wang, Z. Cai, J. Liu, J. Li, and X. Zhao, "MADRL-based URLLC-aware task offloading for air-ground vehicular cooperative computing network," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 7, pp. 6716–6729, Jul. 2024.
- [136] X. Mao, G. Wu, M. Fan, Z. Cao, and W. Pedrycz, "DL-DRL: A double-level deep reinforcement learning approach for large-scale task scheduling of multi-UAV," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 1028–1044, 2025.
- [137] F. Yang et al., "A federated reinforcement learning approach for optimizing wireless communication in UAV-enabled IoT network with dense deployments," *IEEE Internet Things J.*, vol. 11, no. 20, pp. 33953–33966, Oct. 2024.
- [138] Y. Hou, J. Zhao, R. Zhang, X. Cheng, and L. Yang, "UAV swarm cooperative target search: A multi-agent reinforcement learning approach," *IEEE Trans. Intell. Veh.*, vol. 9, no. 1, pp. 568–578, Jan. 2024.
- [139] Z. Hu, X. Gao, K. Wan, Q. Wang, and Y. Zhai, "Asynchronous curriculum experience replay: A deep reinforcement learning approach for UAV autonomous motion control in unknown dynamic environments," *IEEE Trans. Veh. Technol.*, vol. 72, no. 11, pp. 13985–14001, Nov. 2023.
- [140] E. Böhn, E. M. Coates, D. Reinhardt, and T. A. Johansen, "Data-efficient deep reinforcement learning for attitude control of fixed-wing UAVs: Field experiments," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 3, pp. 3168–3180, Mar. 2024.



Wei Zhao (Member, IEEE) received the Ph.D. degree from the Graduate School of Information Sciences, Tohoku University. He is currently a Professor with the School of Computer Science and Technology, Anhui University of Technology. His research interests include deep reinforcement learning, edge computing, and resource allocation in wireless networks. He was the recipient of the IEEE WCSP-2014 Best Paper Award and the IEEE GLOBECOM-2014 Best Paper Award.



Shaoxin Cui (Graduate Student Member, IEEE) is currently pursuing the master's degree in engineering with the Anhui University of Technology. His research interests center around deep reinforcement learning and edge computing, specifically in their application to unmanned aerial vehicles (UAVs). By leveraging advanced computing techniques, he aims to enhance UAV decision-making processes and overall performance for various applications.



Wen Qiu (Member, IEEE) received the Ph.D. degree in co-creative engineering from the Kitami Institute of Technology, Hokkaido, Japan. She is currently a Research Assistant with the Anhui University of Technology. Her research interests encompass deep reinforcement learning and emergency wireless communication networks, focusing on developing intelligent networking solutions for disaster response scenarios. Her work specifically addresses challenges in network resilience, dynamic resource allocation, and system optimization using advanced

machine learning techniques.



Xiao Zheng received the Ph.D. degree in computer science and technology from Southeast University, China, in 2014. He is currently a Professor with the School of Computer Science and Technology, Anhui University of Technology, Ma'anshan, Anhui, China. His research interests include service computing, mobile cloud computing, and privacy protection. He has been a Guest Editor of *IEICE Transactions on Communications*. He is a Senior Member of CCF, and a member of ACM.



Zhiqiang He (Member, IEEE) received the M.S. degree in control science and engineering from Northeastern University, Shenyang, China. He is currently a Research Assistant with the Anhui University of Technology. He previously worked with Baidu and InspirAI, where he developed a master-level AI for the game Landlord that outperformed professional players. His research interests focus on deep reinforcement learning and its control applications.



Bomin Mao (Senior Member, IEEE) is currently a Professor with the School of Cybersecurity, Northwestern Polytechnical University, China. His research interests are involving satellite networks, Internet of Things, edge computing, and vehicular networks. He received several Best Paper Awards from international conferences, namely IEEE GLOBECOM'17, GLOBECOM'18, IC-NIDC'18, ICC'23, and WOCC'23. He was a recipient of the prestigious IEEE COMSOC Asia Pacific Outstanding Paper Award in 2020, the Niwa

Yasujiro Outstanding Paper Award in 2019, and the IEEE Computer Society Tokyo/Japan Joint Local Chapters Young Author Award in 2020.



Zhi Liu (Senior Member, IEEE) received the Ph.D. degree in informatics from the National Institute of Informatics. He is currently an Associate Professor with the University of Electro-Communications. His research interests include video network transmission and MEC. He is currently an Editorial Board Member of IEEE TRANSACTIONS ON MULTIMEDIA, IEEE NETWORK, and *Internet of Things Journal*.



Nei Kato (Fellow, IEEE) is a Full Professor and the Dean of the Graduate School of Information Sciences, Tohoku University. He has published more than 500 papers in prestigious peer-reviewed journals and conferences. He has been engaged in research on computer networking, wireless mobile communications, satellite communications, ad hoc and sensor and mesh networks, smart grid, AI, IoT, big data, and pattern recognition. He was the Vice-President (Member and Global Activities) of IEEE Communications Society from 2018 to 2019, and the Editor-in-Chief of IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY from 2017 to 2020, and of IEEE NETWORK from 2015 to 2017. He is a Fellow of the Engineering Academy of Japan and of IEICE.